



Digital Systems Design Assignment

ES2E3 Report

Department of Engineering

u5529041, Aaditya Nair

January 11, 2025

Abstract

This project focuses on the development of an interactive zombie survival game using Verilog, focusing on hardware-based game logic implementation and visual rendering. Using concepts such as finite-state machines, modular design, and timing synchronization, the system integrates game mechanics, including sprite animations, collision detection, and dynamic scoring. The core challenges addressed include efficient resource management, accurate synchronization of game elements, and real-time response to player input. This project provides an engaging exploration of hardware-based gaming, illustrating the potential of FPGA-based systems to implement complex interactive applications.

Contents

1	Introduction	3
2	Approach	4
2.1	Game Top - Player Movement	4
2.2	Drawcon - Sprite Creation and UI components	4
2.3	Zombie - Zombie Movement	5
2.4	Bullet Logic	5
2.5	Clock and Timing	6
3	Extra Features	6
3.1	BDM Mic Interface	6
3.2	Easter Eggs	6
4	System Workflow	7
5	Testing and Results	8
5.1	Game Features and PDM Mic Interface Testing	8
5.2	Player Detection Waveforms	9
5.3	Bullet Position Testing	9
6	Conclusion	10
7	Appendix: Supporting Information	11
8	Verilog Code Files	12
8.1	game_top.v	12
8.2	drawcon.v	18
8.3	zombie.v	39
8.4	bullet_logic.v	47
8.5	pdm_mic_interface.v	54
8.6	clk_conv.v	58
8.7	vga.v	59
8.8	random_number.v	61
8.9	bullet_test.v	62
8.10	sound_tb.v	65
8.11	coe_to_image_convertor	68

1 Introduction

The VGA standard, introduced by IBM in 1987, remains widely used for connecting display systems. It operates at resolutions up to 640x480 at 60 Hz in its base configuration but can be extended to support higher resolutions such as 1440x900, used in this project. VGA transmits video signals through analog means using a 15-pin connector and requires synchronized signaling to ensure proper display rendering.

A VGA display relies on 3 primary signals to render images:

- **Synchronization Signals:** These include horizontal sync (HSYNC) and vertical sync (VSYNC), which control the timing of rows and frames, ensuring pixels are displayed in their correct positions.
- **Color Signals:** Three analog color channels—red, green, and blue (RGB)—determine the color of each pixel. The intensity of these signals varies between 0 and 0.7V to define the pixel's shade.
- **Control Timings:** The VGA controller must follow precise timing standards to display a stable image. For 1440x900 resolution, the controller must account for both the active display area and the blanking intervals required for the generation of the sync signal.

The VGA controller in this project manages these signals using a pixel clock, ensuring the resolution (1440x900) is properly displayed while maintaining synchronization with the game's logic.

In "Project Deathless Dusk," the VGA display is crucial for rendering the 720x440 game map on a 1440x900 screen. This involves scaling and centering the map while ensuring collision maps, sprites, and animations are visually coherent. The game employs features such as:

- **Collision detection maps:** Using a grid of 1s and 0s, collision areas are superimposed on the background to restrict movements of the player, zombies, and bullets.
- **Sprites and Animations:** Sprite movements and animations are synchronized with the game's clocks, enhancing visual fluidity.
- **User Inputs:** The VGA controller interfaces with the board's buttons and switches to manage player movements and weapon switching.

To achieve this, block memory generators in Verilog are utilized to store and retrieve data efficiently, while a clock wizard generates the slower clocks necessary for sprite movements and game updates.

This project explores the process of designing and implementing a game using combinational and sequential circuits, using Verilog for hardware description and prototyping on the Nexys 4 FPGA board. The primary objective of this project is to create a functional and engaging game that not only demonstrates the application of theoretical concepts in digital systems but also highlights the integration of hardware and software in game development.

2 Approach

The game is split into five waves where each wave has an increased level of difficulty. The player can proceed to the next stage only after defeating every active zombie. The approach to developing the game was carefully structured into multiple stages, each focusing on a specific aspect of the design and implementation process.

2.1 Game Top - Player Movement

The game top module serves as the central control unit of the system integrating various subsystems to facilitate game functionalities such as player movement, collision detection, visual output, and interaction with external components like sound sensors and displays. It connects multiple submodules, including the drawing controller (drawcon), sound processor (pdm_mic_interface), VGA controller, and collision detection unit. The module operates on a hierarchical clock system to manage different tasks efficiently.

The player movement is controlled by the buttons (`btn`) on the board. The buttons `btn[1:4]` control the direction the player is moving in, while `btn [0]` controls the animation of the shooting when the player is stationary. If the movement buttons and the shooting button is pressed together, it allows the player to move at two times their original speed. The player utilizes the `zombie_attack_total` register for displaying injured frames, which computes the combined attack of all the zombie's within the player's reach and highlights the player red incase the value is 1. The collision detection uses memory address to check the collision status of every point on the map. These collision points are designed using a collision map which has been resized to 45x27 to depict every 16x16 pixel cell. This map is designed in a binary format where 1 represents a collision point and 0 represents a walkable region. Using `player_loc_x` and `player_loc_y` which represents simplified discrete grid coordinates, the unwalkable regions can be detected. The zombies are designed to bypass collision rules, introducing an additional layer of complexity and enhancing the challenge for the player.

A state machine is implemented to pipeline the entire collision-checking process. The `STATE_ADDR` calculates the memory address for the intended movement direction. In order to allow the memory to stabilize, `STATE_WAIT` waits for one clock cycle and then proceeds to the next state. `STATE_READ` reads the collision data to check if a collision is detected and then blocks the movement by setting up the `collision` register. Finally, `STATE_UPDATE` updates the player's position if no collision is detected and sets the movement direction.

2.2 Drawcon - Sprite Creation and UI components

The drawcon module consisted of drawing UI elements to add to the VGA interface. The UI components run at `pixclk` so that the resolution remains constant. The maps as well as the sprites were developed on [Piskel](#). The map (720x440) contains various components which function in different ways. Certain areas of the map, such as the tree trunks, river, and bunker walls, include collision points designed to prevent both the player and zombie sprites from passing through them.

Certain locations, such as the bridge and the bunker entrance near the graveyard at the top, are designed as strategic entry points for players. These areas provide safe

zones where players can retreat when overwhelmed by zombies. To maintain balanced gameplay, these points are inaccessible to zombies, and players are restricted from shooting through them. New zombies are programmed to spawn near the bunker, whereas the defeated zombies from previous waves respawn at their original positions, creating a visual "reviving" effect and adding to the challenge. A few components contain various frames which change at different speeds and conditions. The player and zombies updates on `sprtclk` and updates its frame during movement every clock cycle. The other components update based on a condition. For example the wave animation displays different frames based on the `wave_counter`.

Each UI component is equipped with address registers that store its location. The `rom_pixel` values for each element are retrieved from block generator instances and subsequently passed to these registers, which then feed into the printing logic. The sound UI contains different stages which is printed depending on the omnidirectional audio output from the mic as well as the sound created by the player sprinting or the noise produced by the guns. Different guns have different variations in noise. For example, the shotgun, produces a louder noise compared to the knife. The drawcon file also computes the scoring logic. A score counter is used to keep track of the total number of zombies that have been killed every wave.

2.3 Zombie - Zombie Movement

Each zombie contains a detection radius that detects whether the player is nearby or not. This is calculated using the absolute difference between the zombie and the player. When the player is not within this zone, the zombie is assigned a random direction using `dir`, which it follows, so that the zombie is never stationary. When the player enters the zombie's detection radius, the zombie actively tracks the player's position, determined by `blkpos_x` and `blkpos_y`, and initiates an attack when the player comes within its designated attack range. The `sound_state` serves as a mechanism to detect the player's sprite movement and generated noise, such as shooting, and prompts the zombies to pursue the player until they move outside the detection radius. This state is influenced by both in-game sounds and input from the omnidirectional microphone, which makes the game more immersive and interactive. If the player injures any of the zombies, the `hit_timer` register sends a signal to the printing logic which highlights the zombie red.

Once a zombie has been defeated it remains dead until the next wave where it respawns. The zombie will still detect the player's location for 255 `gameclk` cycles after being shot, to show the zombie is still partially alive. The number of zombies increase every wave by 1 and so does its health which is controlled by `hit_counter`.

2.4 Bullet Logic

The game allows players to switch between weapons with unique functionalities using the switches on the board, with the assault rifle set as the default weapon. The assault rifle has a speed of approximately 12 bullets per second. The shotgun and the throwing knife has a fire-rate of 0.6 bullets per second, whereas the pistol is semi-automatic. The bullets are designed such that they do not pass through the bunker walls and the river which highlights the collision logic for the bullets. The location and direction of each bullet is stored using the appropriate registers to track whether they hit the zombie which

are updated dynamically using the `next_bullet` counter. To maintain balanced gameplay, guns are equipped with a feature called the `bullet_lifetime` which represents the range of the bullets.

2.5 Clock and Timing

Three different clocks are used in the game to ensure that the interface runs smoothly. Apart from the main clock, the game uses the `pixclk` for rendering the game animations which runs at the speed of 106.49 MHz and is generated using the `clk_wiz` generator. The `pixclk` is passed to the `drawcon` module which contains the main logic for printing all the UI elements. For all the game updates a `game_clk` is generated which runs at 60 fps so that all the changes remain consistent. In order to detect collision we use a slower clock (`game_clk_three`) The sprites are updated using the `sprtcclk` which is generated by passing the `pixclk` into the `clk_conv` which produces a slower clock at the speed of 10.649 MHz to process the sprite animations clearly.

3 Extra Features

3.1 BDM Mic Interface

Due to our games focus on noise detection, and the lack of a second player necessitating additional controls such as a keyboard, we decided to use the ADI ADMP42' PDC mic. Using the [reference manual](#), we were able to create an algorithm that sums up and output all the bits within a pulse of frequency 2.5Mhz. As we had no grasp of the threshold values we decided to connect the output values to a seven bit segment, which would display the `pcm_data`. Initially it did not output anything, however we saw flickers when extended noise was applied to the system. This was due to the microphone stored output being overwritten immediately after it was updated, consequently we added a buffer that stored the largest signal within a 3 second time period. The seven segment proved to be a good measure of noise as shouting at a reasonable distance could not exceed the value of 4, however under extreme conditions (such as a snapping finger) the values can overflow, however this is an edge case and is irrelevant to casual game play.

3.2 Easter Eggs

The game is equipped with a few easter eggs to improve gameplay. If the player ventures to the zombie spawn area and passes through the gate, they can enter the bunker. Both the bunker and the starting area are designated as safe zones where the player is protected from zombies. However, while the player can shoot outside the bunker, shooting is disabled in the starting area. This design choice ensures fair gameplay by preventing the player from exploiting safe zones to win the game.

Another exciting easter egg in the game is a unique transition mechanic between the shotgun and the knife. If the player switches weapons from the shotgun to the knife immediately after firing, the shotgun bullets are transformed into knives mid-flight. This hidden feature adds a surprising and entertaining twist to the gameplay.

4 System Workflow

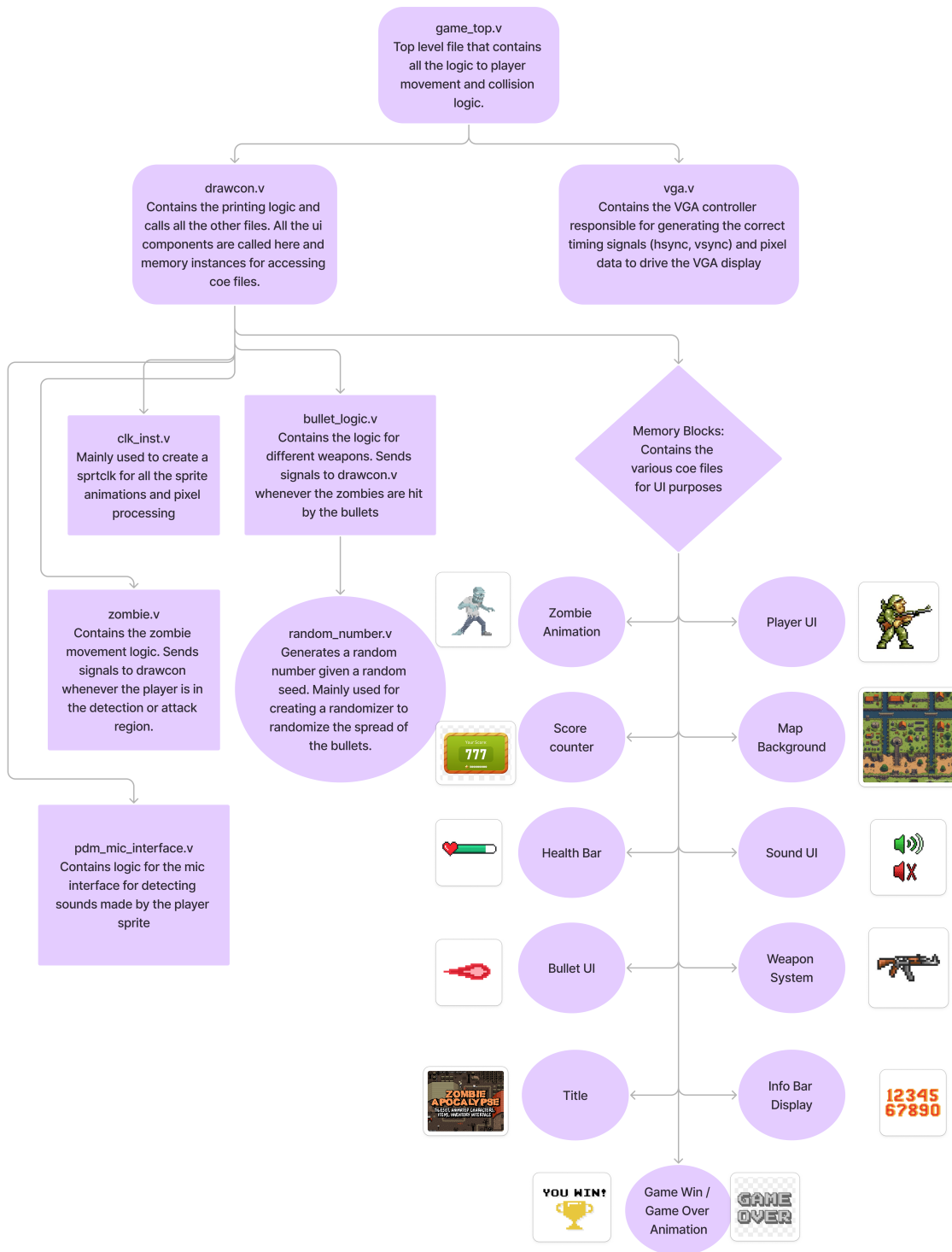
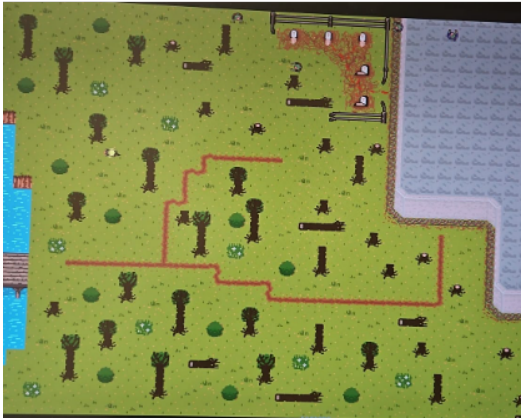


Figure 1: System Workflow Diagram: The overall architecture of the project, showing file dependencies, game logic, and the VGA controller's role in rendering UI components and animations.

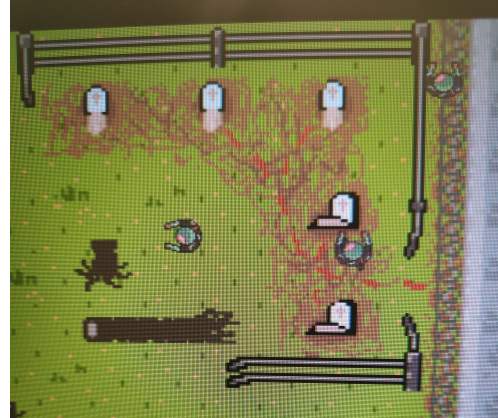
5 Testing and Results

5.1 Game Features and PDM Mic Interface Testing

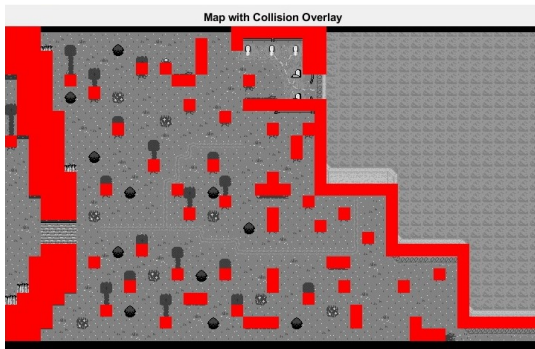
The testing of the collision map involved creating a MATLAB script to superimpose the collision map with the background to adjust the collision points (Section 8.11). This involved using `imresize` and `repmat` function to display a grayscale image of the map with red sections to depict the collision points as shown in Figure 2c. This allowed testing to be conducted seamlessly. The omnidirectional microphone was tested using a 7-segment display to represent different noise intensities. The audio captured by the microphone is processed and passed to the top module, which outputs the corresponding noise levels on the 7-segment display. The sound levels are categorized as follows: Ambient (0–2), Conversational (2–3), and Loud Noise (>3). Figure 2d depicts how the display reacts to various noise levels (loud clapping in this case). The different sound variations are displayed on the sound bar as shown in the video. Various images of the game and testing features are displayed below.



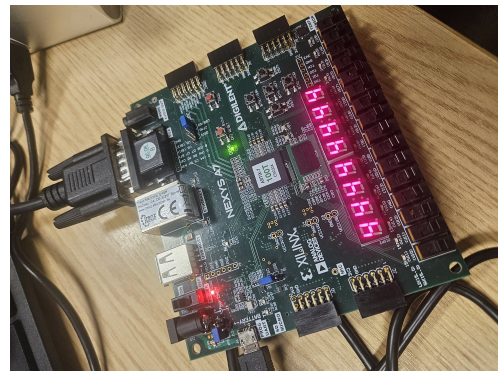
(a) Test Case 1: Initial Setup



(b) Test Case 2: Zombie Respawn



(c) Test Case 3: Collision Detection



(d) Test Case 4: Mic Testing

Figure 2: Results from various test cases in the game development process.

5.2 Player Detection Waveforms

To verify functionality, testbenches were employed to evaluate the detection signal for the zombie instances. Once a player is within the detection radius or if the player produces noise(shooting and movement) the detected signal activates. This can be observed in Figure 3. In this figure, it can be seen that the value of the absolute difference decreases every time the player is detected, which shows how the zombie follows the player.

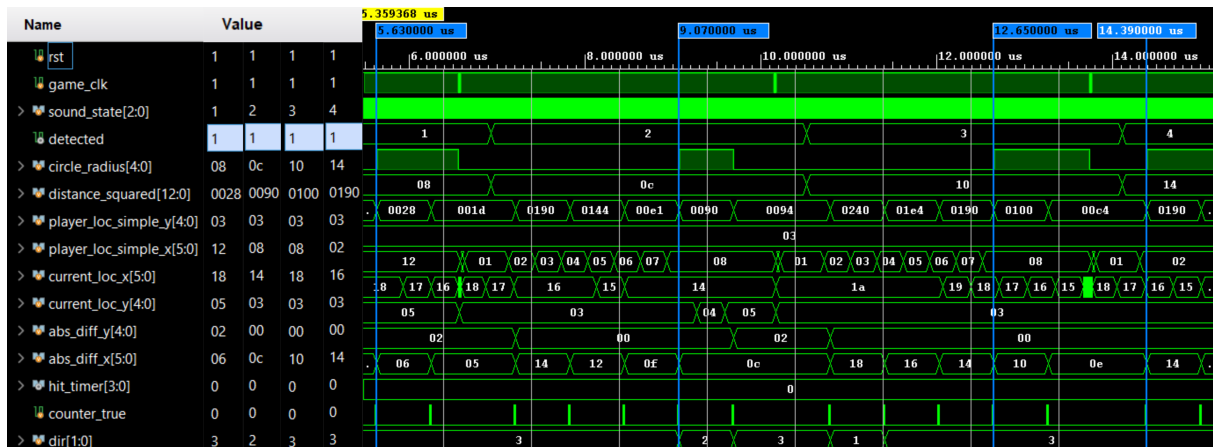


Figure 3: Zombie Simulation Waveform

5.3 Bullet Position Testing

The waveform in Figure 4 demonstrates the behavior of the bullet logic module, including the position updates of bullets (bullet_x, bullet_y) and the changes in their respective velocities over time as they move across the game environment.

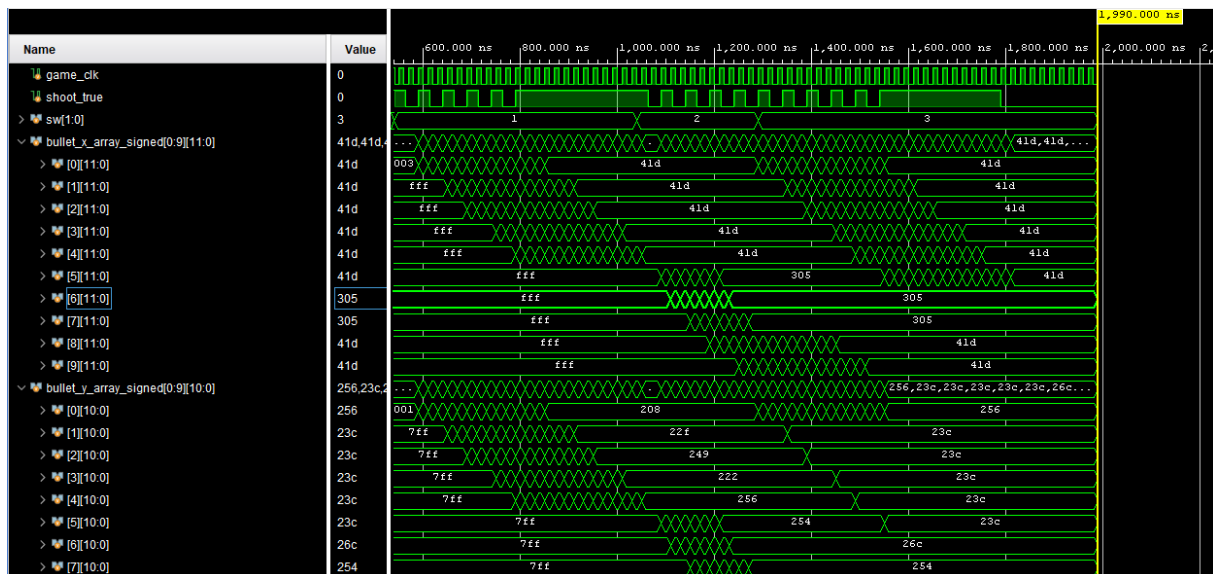


Figure 4: Test Waveform

6 Conclusion

The development of the game "Project Deathless Dusk" provided an in-depth understanding of digital system design principles and their practical implementation using Verilog and FPGA hardware. The integration of various modules, such as the VGA controller for display rendering, game logic for player and zombie interactions, and memory blocks for sprite storage, highlighted the complexity and challenges of creating a cohesive system.

Towards the end, the project was able to successfully implement key features such as collision detection, weapon switching, a dynamic health bar, and sound detection which were rendered smoothly. Testing and debugging were essential aspects of the development process. Using testbenches, we were able to test the functionality of different signals employed in the game as well as test the sound detection using the mic which was built in the game.

Overall, the project was a rewarding experience that enhanced our technical skills and understanding of hardware design. It served as a practical platform to apply theoretical knowledge and opened avenues for further exploration in the fields of game development and digital systems.

7 Appendix: Supporting Information

- **Omnidirectional Microphone Reference Manual:** Comprehensive details about the mic feature used for detecting sound. <https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual#microphone>
- **Clock Generator in Verilog:** A sample of developing a clock generator to design the sprite clock <https://www.chipverify.com/verilog/verilog-clock-generator>
- **FPGA Forum:** Website which allows users to view and ask question relating to FPGA's and Analog Design <https://www.edaboard.com/threads/nexys-4-drr-collision-detection-for-labyrinth-game.401997/>
- **Syntaxing in Verilog:** A detailed guide on programming in Verilog <https://www.asic-world.com/verilog/veritut.html>
- **Storing Images on BRAM:** An article regarding converting images to coe files and uploading them on BRAM blocks <https://www.imperva.com/learn/ddos/syn-flood/>
- **Number Handling and Unsigned Representation in Verilog:** Explains key concepts such as unsigned representation, bit-width specifications, and the behavior of signed and unsigned numbers in arithmetic and logical operations. <https://projectf.io/posts/numbers-in-verilog/#:~:text=Verilog%20doesn't%20consider%20it,treats%20them%20all%20as%20unsigned.>

8 Verilog Code Files

8.1 game_top.v

```
`timescale 1ns / 1ps

module game_top(
    input clk,
    input rst,
    input [1:0] sw,
    input [4:0] btn,
    output [3:0] pix_r,
    output [3:0] pix_g,
    output [3:0] pix_b,
    output hsync,
    output vsync,
    output M_CLK,
    output M_LRSEL,
    input M_DATA,
    output a, b, c, d, e ,f, g
);

wire [6:0] seg;
assign {a, b, c, d, e, f, g} = seg; // Used for displaying the seven segment
    ↪ display for detecting sound

// Internal wires
wire pixclk;
wire [3:0] draw_r;
wire [3:0] draw_g;
wire [3:0] draw_b;
wire [10:0] curr_x;
wire [10:0] curr_y;

reg [20:0] clk_div;
reg game_clk;
reg game_clk_three;

// Player position and direction
reg [10:0] blkpos_x;
reg [9:0] blkpos_y;
reg [1:0] dir;
reg move_animation_true;
reg shoot_true;
wire [2:0] display_wave; // Signals other ui components whenever the wave is
    ↪ changed

// Player simplified coordinates
reg [5:0] player_loc_simple_x;
reg [4:0] player_loc_simple_y;
```

```

// Collision detection
reg [3:0] collision;
reg [10:0] collision_addr;
wire collision_vision;

// Player health counter
wire [2:0] player_health;

reg sample_en;
wire [1:0] pdm_output; // Contains the output of the mic which detects sound

// State machine for pipelined collision checking
reg [2:0] state;
localparam STATE_ADDR = 3'd0;
localparam STATE_WAIT = 3'd1;
localparam STATE_WAIT2 = 3'd2;
localparam STATE_READ = 3'd3;
localparam STATE_UPDATE = 3'd4;

// Direction attempt storage
reg [3:0] direction_input;

//clock generator for pixel clock
clk_wiz inst
(
    .clk_out1(pixclk),
    .clk_in1(clk)
);

// Creates 60fps clock
always @(posedge pixclk or negedge rst) begin
    if(!rst) begin
        clk_div <= 0;
        game_clk <= 0;
        game_clk_three <= 0;
    end else begin
        if (clk_div == 21'd591494 || clk_div == (21'd591494 * 2) || clk_div ==
            ↪ (21'd591494 * 3)) begin
            game_clk_three <= !game_clk_three;
        end
        if ((clk_div == 21'd1774483)) begin
            clk_div <= 0;
            game_clk <= !game_clk;
        end else begin
            clk_div <= clk_div + 1;
        end
    end
end
end

```

```

// Movement and collision checking state machine
always @(posedge game_clk_three or negedge rst) begin
    if(!rst || display_wave) begin
        blkpos_x <= 48;
        blkpos_y <= 560;
        move_animation_true <= 0;
        shoot_true <= 0;
        collision <= 4'b0000;
        direction_input <= 4'b0000;
        state <= STATE_ADDR;
    end else begin
        shoot_true <= btn[0] ? 1'b1 : 1'b0;

        case (state)
            STATE_ADDR: begin
                // We set the collision_addr here but do NOT immediately read
                ↪ collision_vision
                // Reset collision flags for this new check

                collision <= 4'b0000;

                case(btn[4:1]) // was btn before
                    4'b0010: begin // Left
                        direction_input <= 4'b0010;
                        if (player_loc_simple_x > 0)
                            collision_addr <= player_loc_simple_y * 45 +
                                ↪ (player_loc_simple_x - 1);
                        else
                            collision_addr <= player_loc_simple_y * 45 +
                                ↪ player_loc_simple_x;
                    end
                    4'b0100: begin // Right
                        direction_input <= 4'b0100;
                        if (player_loc_simple_x < 44)
                            collision_addr <= player_loc_simple_y * 45 +
                                ↪ (player_loc_simple_x + 1);
                        else
                            collision_addr <= player_loc_simple_y * 45 +
                                ↪ player_loc_simple_x;
                    end
                    4'b1000: begin // Up
                        direction_input <= 4'b1000;
                        if (player_loc_simple_y > 0)
                            collision_addr <= (player_loc_simple_y + 1) * 45 +
                                ↪ player_loc_simple_x;
                        else
                            collision_addr <= player_loc_simple_y * 45 +
                                ↪ player_loc_simple_x;
                    end
                    4'b0001: begin // Down
                        direction_input <= 4'b0001;

```

```

        if (player_loc_simple_y < 27)
            collision_addr <= (player_loc_simple_y - 1) * 45 +
                ↪ player_loc_simple_x;
        else
            collision_addr <= player_loc_simple_y * 45 +
                ↪ player_loc_simple_x;
        end
    default: begin
        direction_input <= 4'b0000;
        collision_addr <= player_loc_simple_y * 45 +
            ↪ player_loc_simple_x;
    end
endcase

state <= STATE_WAIT;
end

STATE_WAIT: begin
    // Just wait one game_clk cycle here.
    // The collision memory is synchronous with 'clk',
    // by the time we hit next state (READ), collision_vision should
    ↪ be stable.
    state <= STATE_READ;
end

// Reads from the collision data
STATE_READ: begin
    if (collision_vision == 1'b1) begin
        case(direction_input)
            4'b0010: collision[3] <= 1; // Left
            4'b0100: collision[1] <= 1; // Right
            4'b1000: collision[2] <= 1; // Up
            4'b0001: collision[0] <= 1; // Down
            default: collision <= 4'b0000;
        endcase
    end
    state <= STATE_UPDATE;
end

// Checks for collision and updates player movement
STATE_UPDATE: begin
    // Attempt movement if no collision occurred
    if (direction_input != 4'b0000) begin
        case (direction_input)
            4'b0010: begin // Left
                dir <= 2'd3;
                if (blkpos_x > 11'd10 && !collision[3]) begin
                    $display("Moving Left: New X = %d", blkpos_x -
                        ↪ 2 - 2 * (shoot_true + 1));
                    blkpos_x <= blkpos_x - 2 - 2 * (shoot_true + 1);
                end
            end
        endcase
    end
end

```

```

        move_animation_true <= 1;
    end else begin
        move_animation_true <= 0;
    end
end
4'b0100: begin // Right
    dir <= 2'd1;
    if (blkpos_x < (11'd1430 - 11'd50) &&
        !collision[1]) begin
        $display("Moving Right: New X = %d", blkpos_x +
            2 + 2 * (shoot_true + 1));
        blkpos_x <= blkpos_x + 2 + 2 * (shoot_true + 1);

        move_animation_true <= 1;
    end else begin
        move_animation_true <= 0;
    end
end
4'b1000: begin // Up
    dir <= 2'd0;
    if (blkpos_y < (11'd890 - 11'd50) && !collision[2])
        begin
        $display("Moving Up: New Y = %d", blkpos_y + 2
            + 2 * (shoot_true + 1));
        blkpos_y <= blkpos_y + 2 + 2 * (shoot_true + 1);

        move_animation_true <= 1;
    end else begin
        move_animation_true <= 0;
    end
end
4'b0001: begin // Down
    dir <= 2'd2;
    if (blkpos_y > 11'd10 && !collision[0]) begin
        $display("Moving Down: New Y = %d", blkpos_y -
            2 - 2 * (shoot_true + 1));
        blkpos_y <= blkpos_y - 2 - 2 * (shoot_true + 1);

        move_animation_true <= 1;
    end else begin
        move_animation_true <= 0;
    end
end
default: move_animation_true <= 0;
endcase
end else begin
    move_animation_true <= 0;
end

// Go back to STATE_ADDR to handle next input
state <= STATE_ADDR;

```

```

        end
    endcase
end
end

// Creates a player loc simple for zombie tracking
always @(posedge game_clk_three or negedge rst) begin
    if(!rst || display_wave) begin
        player_loc_simple_x <= 6'd0;
        player_loc_simple_y <= 2'd0;
    end else begin
        player_loc_simple_x <= blkpos_x / 32;
        if (blkpos_y > 16) begin
            player_loc_simple_y <= (blkpos_y-16) / 32;
        end else begin
            player_loc_simple_y <= 0;
        end
    end
end
end

assign M_LRSEL = 1;

pdm_mic_interface pdm_mic_interface_inst(
    .clk(clk),
    .rst(rst),
    .pdm_data(M_DATA),
    .pdm_clk(M_CLK),
    .pdm_output(pdm_output),
    .seg(seg),
    .game_clk_three(game_clk_three)
);

drawcon drawcon_inst(
    .clk(pixclk),
    .rst(rst),
    .sw(sw),
    .blkpos_x(blkpos_x),
    .blkpos_y(blkpos_y),
    .draw_r(draw_r),
    .draw_g(draw_g),
    .draw_b(draw_b),
    .curr_x(curr_x),
    .curr_y(curr_y),
    .dir(dir),
    .move_animation_true(move_animation_true),
    .shoot_true(shoot_true),
    .game_clk(game_clk),
    .player_loc_simple_x(player_loc_simple_x),
    .player_loc_simple_y(player_loc_simple_y),

```

```

        .display_wave(display_wave),
        .pdm_output(pdm_output)
    );

vga vga_inst(
    .clk(pixclk),
    .rst(rst),
    .draw_r(draw_r),
    .draw_g(draw_g),
    .draw_b(draw_b),
    .pix_r(pix_r),
    .pix_g(pix_g),
    .pix_b(pix_b),
    .curr_x(curr_x),
    .curr_y(curr_y),
    .hsync(hsync),
    .vsync(vsync)
);

blk_mem_gen_8 Collision_data_player
(
    .clka(pixclk),
    .addra(collision_addr),
    .douta(collision_vision)
);

endmodule

);
    // Your Verilog code here
endmodule

```

8.2 drawcon.v

```

`timescale 1ns / 1ps

module drawcon(
    input clk,
    input rst,
    input [1:0] sw,
    input [10:0] blkpos_x,
    input [9:0] blkpos_y,
    output [3:0] draw_r,
    output [3:0] draw_g,
    output [3:0] draw_b,
    input [10:0] curr_x,
    input [10:0] curr_y,

    input [1:0] dir,
    input move_animation_true,

```

```

input shoot_true,
input game_clk,
input [5:0] player_loc_simple_x, // map divided into 16 * 16 squares 26*46
input [4:0] player_loc_simple_y,
output reg [2:0] display_wave,
input [1:0] pdm_output
// input [2:0] player_health

);

reg [3:0] bg_r;
reg [3:0] bg_g;
reg [3:0] bg_b;

reg [3:0] sprite_counter;
reg [1:0] sprite_direction;
reg [2:0] sprite_animation_stage;

reg [3:0] blt_r;
reg [3:0] blt_g;
reg [3:0] blt_b;

reg [3:0] print_r;
reg [3:0] print_g;
reg [3:0] print_b;

parameter BLK_SIZE_X = 33, BLK_SIZE_Y = 20;
parameter AREA = 660;
parameter PIXEL_SCALE = 1;
parameter BULLET_SIZE = 4;
parameter NUM_ZOMBIES = 8;

parameter GAME_OVER_SIZE_X = 36;
parameter GAME_OVER_SIZE_Y = 22;
parameter GAME_OVER_PIXEL_SCALE = 10; // Scale factor
parameter GAME_OVER_X_START = (1440 - GAME_OVER_SIZE_X *
↳ GAME_OVER_PIXEL_SCALE) / 2; // 540
parameter GAME_OVER_Y_START = (900 - GAME_OVER_SIZE_Y *
↳ GAME_OVER_PIXEL_SCALE) / 2 - 15; // 340
parameter GAME_OVER_AREA = GAME_OVER_SIZE_X * GAME_OVER_SIZE_Y;

parameter WAVE_SIZE_X = 16;
parameter WAVE_SIZE_Y = 16;
parameter WAVE_PIXEL_SCALE = 10;
parameter WAVE_X_START = (1440 - WAVE_SIZE_X * WAVE_PIXEL_SCALE) / 2;
parameter WAVE_Y_START = (900 - WAVE_SIZE_Y * WAVE_PIXEL_SCALE) / 2;
parameter WAVE_AREA = WAVE_SIZE_X * WAVE_SIZE_Y;

parameter TITLE_SIZE_X = 48;
parameter TITLE_SIZE_Y = 20; // 1 Less than in the files since the bottom is
↳ a black row

```



```

parameter TITLE_PIXEL_SCALE = 2;
parameter TITLE_X_START = (1440 - TITLE_SIZE_X * TITLE_PIXEL_SCALE)/2 ; //
↳ [672 768]
parameter TITLE_Y_START = (900 - TITLE_SIZE_Y * TITLE_PIXEL_SCALE); // [853
↳ 895]
parameter TITLE_AREA = TITLE_SIZE_X * TITLE_SIZE_Y;

parameter SKULL_SIZE_X = 15;
parameter SKULL_SIZE_Y = 14;
parameter SKULL_PIXEL_SCALE = 2;
parameter SKULL_X_START = (1440 - SKULL_SIZE_X * SKULL_PIXEL_SCALE) - 56 ;
↳ // [1310 1340]
parameter SKULL_Y_START = (900 - SKULL_SIZE_Y * SKULL_PIXEL_SCALE) - 5; //
↳ [867 895]
parameter SKULL_AREA = SKULL_SIZE_X * SKULL_SIZE_Y;

parameter ID_SIZE_X = 40;
parameter ID_SIZE_Y = 14;
parameter ID_PIXEL_SCALE = 2;
parameter ID_X_START = (1440 - ID_SIZE_X * ID_PIXEL_SCALE)/2 + 300 ; // [778
↳ 858]
parameter ID_Y_START = (900 - ID_SIZE_Y * ID_PIXEL_SCALE) - 4; // [867 895]
parameter ID_AREA = ID_SIZE_X * ID_SIZE_Y;

parameter OUTPUT_SIZE_X = 7;
parameter OUTPUT_SIZE_Y = 11;
parameter OUTPUT_PIXEL_SCALE = 2;
parameter OUTPUT_X_START = (1440 - OUTPUT_SIZE_X * OUTPUT_PIXEL_SCALE) - 20
↳ ; // [1344 1440]
parameter OUTPUT_Y_START = (900 - OUTPUT_SIZE_Y * OUTPUT_PIXEL_SCALE) - 10;
↳ // [853 895]
parameter OUTPUT_X_START_2 = (1440 - OUTPUT_SIZE_X * OUTPUT_PIXEL_SCALE) -
↳ 20 - OUTPUT_SIZE_X * OUTPUT_PIXEL_SCALE - 4; // 4 Pixel gap
parameter OUTPUT_Y_START_2 = (900 - OUTPUT_SIZE_Y * OUTPUT_PIXEL_SCALE) -
↳ 10;
parameter OUTPUT_AREA = OUTPUT_SIZE_X * OUTPUT_SIZE_Y;

wire zombie_attack [0:NUM_ZOMBIES-1]; // array containing the zombie_attack
↳ value for each zombie
reg zombie_attack_total; // 1 if the zombie attacks or 0 otherwise
reg [3:0] active_zombies; // maximum number of zombies in a wave

wire detected [0:NUM_ZOMBIES-1]; // Passed through the zombie instance if
↳ the player is within the detection radius

wire [3:0] hit_timer [0:NUM_ZOMBIES-1]; // produces a delay between zombie
↳ attacks

wire [11:0] rom_pixel;
reg [11:0] rom_pixel_reg;

```

```

reg [10:0] bullet_x_arr [0:9];
reg [9:0] bullet_y_arr [0:9];
wire [109:0] bullet_x;
wire [99:0] bullet_y;

reg [3:0] score; // contains the number of zombies defeated every wave
parameter SCORE = 9;

wire [11:0] rom_pixel_background;
reg [11:0] rom_pixel_background_reg;

// Weapons UI reg
reg [11:0] ui_addr_reg;
reg [11:0] rom_pixel_ui_reg;
wire [11:0] rom_pixel_ui;

// Health UI reg
reg [12:0] health_ui_addr_reg;
reg [11:0] rom_pixel_health_ui_reg;
wire [11:0] rom_pixel_health_ui;
reg [2:0] health_state; // For switching between different health levels of
    → the health bar
reg [6:0] health_timer; // Makes the health bar wait for 2.5 seconds before
    → updating
reg game_over; // Prevents player input when this flag is raised
reg game_win; // 1 if the player wins the game

// Sound UI reg
reg [12:0] sound_ui_addr_reg;
reg [11:0] rom_pixel_sound_ui_reg;
wire [11:0] rom_pixel_sound_ui;
reg [2:0] sound_state;
reg [2:0] sound_state_buffer;
reg [2:0] sound_state_buffer_gun;

reg injure_player; // 1 if the player has been injured by the zombie

parameter STATE_CALCULATE = 0;
parameter STATE_UPDATE = 1;

// Zombie UI reg
reg [11:0] rom_pixel_zombie_ui_reg [0:NUM_ZOMBIES-1];
wire [12:0] zombie_computed_reg [0:NUM_ZOMBIES-1]; // Outputs from zombies
wire [11:0] rom_pixel_zombie_ui [0:NUM_ZOMBIES-1];
wire [12:0] zombie_computed_addr;

wire shoot_flag; // Indicates the player is shooting

// Wave UI reg
reg [11:0] wave_ui_addr_reg;

```

```

reg [11:0] rom_pixel_wave_ui_reg;
wire [11:0] rom_pixel_wave_ui;
reg [3:0] wave_state;
reg [3:0] wave_counter; // Keeps track of the current wave
reg [3:0] wave_end_state;
reg [31:0] wave_timer; // Timer to count clock cycles

// Title animation
reg [9:0] title_ui_addr_reg;
reg [11:0] rom_pixel_title_ui_reg;
wire [11:0] rom_pixel_title_ui;

// Skull animation
reg [7:0] skull_ui_addr_reg;
reg [11:0] rom_pixel_skull_ui_reg;
wire [11:0] rom_pixel_skull_ui;

// Student id animation
reg [9:0] id_ui_addr_reg;
reg [11:0] rom_pixel_id_ui_reg;
wire [11:0] rom_pixel_id_ui;

// Output animation
reg [9:0] output_ui_addr_reg;
reg [11:0] rom_pixel_output_ui_reg;
wire [11:0] rom_pixel_output_ui;
reg [9:0] output_ui_addr_reg_2;
reg [11:0] rom_pixel_output_ui_reg_2;
wire [11:0] rom_pixel_output_ui_2;

parameter SPRTCLK_FREQ = 11; // Approximate frequency of sprtclk in Hz
parameter ONE_MINUTE_CYCLES = 60 * SPRTCLK_FREQ; // Number of sprtclk cycles
↳ for 1 minute

// End screen UI
reg [12:0] end_screen_ui_addr_reg;
reg [11:0] rom_pixel_end_screen_ui_reg;
wire [11:0] rom_pixel_end_screen_ui;
reg [3:0] end_screen_state;

reg [12:0] animation_counter; // Counter for slowing down updates
parameter ANIMATION_DELAY = 10; // Adjust this value for desired delay on
↳ wave

// 10fps clk
wire sprtclk;
wire new_clk;

reg [2:0] game_over_counter;

```

```

integer i;
genvar j;
genvar k;
genvar m;

reg [5:0] kill_count; // Keeps track of the total number of zombies killed

wire dead [0:NUM_ZOMBIES-1]; // Contains 1 or 0 which determine if the
    ↪ zombie is alive or not

// Compute sprite address combinationaly
reg [12:0] computed_addr;

reg [3:0] score_buffer;
reg [3:0] score_buffer_squared;

reg [1:0] end_screen_stage;
// ADDRESS CALCULATIONS

// Soldier sprite address calculator (with rotation)
always @* begin
    computed_addr = 0;
    // Determine address based on sprite direction and position
    if (sprite_direction == 2'd1 || sprite_direction == 2'd3) begin
        if (blkpos_x <= curr_x && curr_x <= blkpos_x + BLK_SIZE_X *
            ↪ PIXEL_SCALE - 1 &&
            blkpos_y <= curr_y && curr_y <= blkpos_y + BLK_SIZE_Y *
            ↪ PIXEL_SCALE - 1) begin

            case (sprite_direction)
                2'd1:
                    computed_addr = sprite_animation_stage * AREA
                        + ((curr_y - blkpos_y)/PIXEL_SCALE) *
                        ↪ BLK_SIZE_X
                        + ((curr_x - blkpos_x)/PIXEL_SCALE);
                2'd3:
                    computed_addr = sprite_animation_stage * AREA
                        + (AREA - 1)
                        - (((curr_y - blkpos_y)/PIXEL_SCALE) *
                        ↪ BLK_SIZE_X
                        + ((curr_x - blkpos_x)/PIXEL_SCALE));
            default:
                computed_addr = sprite_animation_stage * AREA
                    + ((curr_y - blkpos_y)/PIXEL_SCALE) *
                    ↪ BLK_SIZE_X
                    + ((curr_x - blkpos_x)/PIXEL_SCALE);
            endcase
        end
    end else if (sprite_direction == 2'd0 || sprite_direction == 2'd2) begin

```

```

if (blkpos_x <= curr_x && curr_x <= blkpos_x + BLK_SIZE_Y *
    ↪ PIXEL_SCALE - 1 &&
    blkpos_y <= curr_y && curr_y <= blkpos_y + BLK_SIZE_X *
    ↪ PIXEL_SCALE - 1) begin

    case (sprite_direction)
        2'd0:
            computed_addr = sprite_animation_stage * AREA
                          + ((curr_x - blkpos_x)/PIXEL_SCALE) *
                          ↪ BLK_SIZE_X
                          + ((curr_y - blkpos_y)/PIXEL_SCALE);

        2'd2:
            computed_addr = sprite_animation_stage * AREA
                          + ((curr_x - blkpos_x)/PIXEL_SCALE) *
                          ↪ BLK_SIZE_X
                          + ((BLK_SIZE_X - 1 - (curr_y -
                          ↪ blkpos_y)/PIXEL_SCALE));

        default:
            computed_addr = sprite_animation_stage * AREA
                          + ((curr_x - blkpos_x)/PIXEL_SCALE) *
                          ↪ BLK_SIZE_X
                          + ((curr_y - blkpos_y)/PIXEL_SCALE);

    endcase
end
end
end

// Checks if zombies are attacking
always @* begin
    zombie_attack_total = 0;
    for (i = 0; i < active_zombies; i = i + 1) begin
        if(zombie_attack[i] == 1 && zombie_attack_total == 0 && !dead[i])
            ↪ begin
                zombie_attack_total = 1;
            end
    end
end

// Weapons ui (first ui so has a default name)
always @* begin
    if (curr_x >= 8 && curr_x < 40 && curr_y >= 873 && curr_y < 895) begin
        // Calculate the offset within the 32x22 UI
        ui_addr_reg = 704 * sw + ((curr_y - 873) * 32) + (curr_x - 8); //
        ↪ Adjust x-coordinate to start at 0
    end else begin
        ui_addr_reg = 0; // Outside the UI area
    end
end

// Health UI
always @* begin

```

```

if (curr_x >= 48 && curr_x < 48 + 64 && curr_y >= 879 && curr_y < 879 +
    ↪ 16) begin
    // Calculate the offset within the 64x16 UI
    health_ui_addr_reg = 1024 * health_state + ((curr_y - 879) * 64) +
    ↪ (curr_x - 48); // Adjust x and y coordinates accordingly
end else begin
    health_ui_addr_reg = 0; // Outside the UI area
    // This lets other sprites overwrite this file when it's null
end
end

// Sound UI
always @* begin
    if (curr_x >= 128 && curr_x < 128 + 64 && curr_y >= 879 && curr_y < 879
    ↪ + 16) begin
        // Calculate the offset within the 64x16 UI
        sound_ui_addr_reg = 1024 * sound_state + ((curr_y - 879) * 64) +
        ↪ (curr_x - 128); // Adjust x and y coordinates accordingly
    end else begin
        sound_ui_addr_reg = 0; // Outside the UI area
        // This lets other sprites overwrite this file when it's null
    end
end

// Wave UI
always @* begin
    if (curr_x >= WAVE_X_START && curr_x < WAVE_X_START + WAVE_SIZE_X *
    ↪ WAVE_PIXEL_SCALE
        && curr_y >= WAVE_Y_START && curr_y < WAVE_Y_START + WAVE_SIZE_Y *
        ↪ WAVE_PIXEL_SCALE) begin
        // Calculate the offset within the 64x16 UI
        wave_ui_addr_reg = wave_state * WAVE_AREA
            + ((curr_y - WAVE_Y_START) / WAVE_PIXEL_SCALE)
            ↪ * WAVE_SIZE_X
            + ((curr_x - WAVE_X_START) / WAVE_PIXEL_SCALE);
    end else begin
        wave_ui_addr_reg = 0; // Outside the UI area
        // This lets other sprites overwrite this file when it's null
    end
end

// End screen UI
always @* begin
    if (curr_x >= GAME_OVER_X_START && curr_x < GAME_OVER_X_START +
    ↪ GAME_OVER_SIZE_X * GAME_OVER_PIXEL_SCALE
        && curr_y >= GAME_OVER_Y_START && curr_y < GAME_OVER_Y_START +
        ↪ GAME_OVER_SIZE_Y * GAME_OVER_PIXEL_SCALE
        && end_screen_state < 6) begin // end_screen_state == 7
        end_screen_ui_addr_reg = end_screen_state * GAME_OVER_AREA

```

```

        + ((curr_y - GAME_OVER_Y_START) /
        ↪ GAME_OVER_PIXEL_SCALE) * GAME_OVER_SIZE_X
        + ((curr_x - GAME_OVER_X_START) /
        ↪ GAME_OVER_PIXEL_SCALE);

    end else begin
        end_screen_ui_addr_reg = 0; // Outside the UI area
    end
end

// Title UI
always @* begin
    if (curr_x >= TITLE_X_START && curr_x < TITLE_X_START + 48 *
    ↪ TITLE_PIXEL_SCALE
    && curr_y >= TITLE_Y_START && curr_y < TITLE_Y_START + 20 *
    ↪ TITLE_PIXEL_SCALE) begin
        title_ui_addr_reg = ((curr_y - TITLE_Y_START) / TITLE_PIXEL_SCALE)
        ↪ * TITLE_SIZE_X
        + ((curr_x - TITLE_X_START) /
        ↪ TITLE_PIXEL_SCALE);

    end else begin
        title_ui_addr_reg = 0; // Outside the UI area
    end
end

// Skull UI
always @* begin
    if (curr_x >= SKULL_X_START && curr_x < SKULL_X_START + 15 *
    ↪ SKULL_PIXEL_SCALE
    && curr_y >= SKULL_Y_START && curr_y < SKULL_Y_START + 14 *
    ↪ SKULL_PIXEL_SCALE) begin
        skull_ui_addr_reg = ((curr_y - SKULL_Y_START) / SKULL_PIXEL_SCALE)
        ↪ * SKULL_SIZE_X
        + ((curr_x - SKULL_X_START) /
        ↪ SKULL_PIXEL_SCALE);

    end else begin
        skull_ui_addr_reg = 0; // Outside the UI area
    end
end

// Student ID UI
always @* begin
    if (curr_x >= ID_X_START && curr_x < ID_X_START + 40 * ID_PIXEL_SCALE
    && curr_y >= ID_Y_START && curr_y < ID_Y_START + 14 * ID_PIXEL_SCALE)
    ↪ begin
        id_ui_addr_reg = ((curr_y - ID_Y_START) / ID_PIXEL_SCALE) *
        ↪ ID_SIZE_X
        + ((curr_x - ID_X_START) / ID_PIXEL_SCALE);
        if (!id_ui_addr_reg) begin
            id_ui_addr_reg = 40; // Prints a silver pixel in the top left
            ↪ since top left is 000 in memory
        end
    end
end

```

```

    end else begin
        id_ui_addr_reg = 0; // Outside the UI area
        // This is one of the only ui's with a non black addr 0 pixel
    end
end

// Score display right 0-9
always @* begin
    if (curr_x >= OUTPUT_X_START && curr_x < OUTPUT_X_START + 7 *
        ↪ OUTPUT_PIXEL_SCALE
    && curr_y >= OUTPUT_Y_START && curr_y < OUTPUT_Y_START + 11 *
        ↪ OUTPUT_PIXEL_SCALE) begin
        output_ui_addr_reg = ((kill_count + score) % 10) * OUTPUT_AREA +
            ↪ ((curr_y - OUTPUT_Y_START) / OUTPUT_PIXEL_SCALE) * OUTPUT_SIZE_X
                + ((curr_x - OUTPUT_X_START) /
                    ↪ OUTPUT_PIXEL_SCALE);
    end else begin
        output_ui_addr_reg = 0; // Outside the UI area
    end
end

// Score display left 00 - 90
always @* begin
    if (curr_x >= OUTPUT_X_START_2 && curr_x < OUTPUT_X_START_2 + 7 *
        ↪ OUTPUT_PIXEL_SCALE
    && curr_y >= OUTPUT_Y_START_2 && curr_y < OUTPUT_Y_START_2 + 11 *
        ↪ OUTPUT_PIXEL_SCALE) begin
        output_ui_addr_reg_2 = ((kill_count + score) / 10) % 10 *
            ↪ OUTPUT_AREA + ((curr_y - OUTPUT_Y_START_2) /
                ↪ OUTPUT_PIXEL_SCALE) * OUTPUT_SIZE_X
                    + ((curr_x - OUTPUT_X_START_2) /
                        ↪ OUTPUT_PIXEL_SCALE);
    end else begin
        output_ui_addr_reg_2 = 0; // Outside the UI area
    end
end

reg [18:0] computed_bg_addr;
// Computes the background address
always @* begin
    if (curr_x < 1440 && curr_y < 890 && curr_y >= 10) begin
        computed_bg_addr = (((curr_y / 2) > 4) ? (curr_y / 2 - 5) : 0) *
            ↪ 720 + (curr_x / 2);
    end else begin
        computed_bg_addr = 0;
    end
end

// END ADDRESS CALCULATIONS

// CLOCK DRIVEN LOGIC

```



```

// Background and bullet logic
// Kept seperate from the other printing logic as this is more complicated
↳ than a reset and massive if statement
always @(posedge clk or negedge rst) begin
    if(!rst) begin
        bg_r <= 4'b0000;
        bg_g <= 4'b0000;
        bg_b <= 4'b0000;

        rom_pixel_background_reg <= 0;
    end else begin

        rom_pixel_background_reg <= rom_pixel_background;

        // Bullets
        blt_r = 4'b0000;
        blt_g = 4'b0000;
        blt_b = 4'b0000;

        // Asigns bullet/knife colour
        for (i = 0; i < 10; i = i + 1) begin
            if(bullet_x_arr[i] != 0) begin
                if (bullet_x_arr[i] <= curr_x && curr_x < bullet_x_arr[i] +
↳ BULLET_SIZE &&
                    bullet_y_arr[i] <= curr_y && curr_y < bullet_y_arr[i] +
↳ BULLET_SIZE) begin
                    if (sw == 2'd3) begin
                        blt_r = 4'b1001; // Knife color
                        blt_g = 4'b1001 ;
                        blt_b = 4'b1001;
                    end else begin
                        blt_r = 4'b1111; // Bullet color
                        blt_g = 4'b1101 ;
                        blt_b = 4'b0001;
                    end
                end
            end
        end

        if(game_over || display_wave || game_win) begin
            bg_r <= rom_pixel_background_reg[11:8] >> 2;
            bg_g <= rom_pixel_background_reg[7:4] >> 2;
            bg_b <= rom_pixel_background_reg[3:0] >> 2;
        end else begin
            // Background from rom_pixel_background_reg
            bg_r <= rom_pixel_background_reg[11:8];
            bg_g <= rom_pixel_background_reg[7:4];
            bg_b <= rom_pixel_background_reg[3:0];
        end
    end
end

```

```

end

// Assigns values rom_pixel values from the animations to the registers
always @(posedge clk or negedge rst) begin
    if(!rst) begin
        // Reset other values
        rom_pixel_reg <= 12'b0;
        rom_pixel_ui_reg <= 12'b0;
        rom_pixel_health_ui_reg <= 12'b0;
        rom_pixel_sound_ui_reg <= 12'b0;
        rom_pixel_wave_ui_reg <= 12'b0;
        rom_pixel_end_screen_ui_reg <= 12'b0;
        rom_pixel_skull_ui_reg <= 12'b0;
        rom_pixel_id_ui_reg <= 12'b0;
        rom_pixel_output_ui_reg <= 12'b0;
        rom_pixel_output_ui_reg_2 <= 12'b0;
        rom_pixel_title_ui_reg <= 12'b0;

        for (i = 0; i < NUM_ZOMBIES; i = i + 1) begin
            rom_pixel_zombie_ui_reg[i] <= 12'b0;
        end
    end else begin
        // Update UI pixel data
        rom_pixel_reg <= rom_pixel;
        rom_pixel_ui_reg <= rom_pixel_ui;
        rom_pixel_health_ui_reg <= rom_pixel_health_ui;
        rom_pixel_sound_ui_reg <= rom_pixel_sound_ui;
        rom_pixel_wave_ui_reg <= rom_pixel_wave_ui;
        rom_pixel_end_screen_ui_reg <= rom_pixel_end_screen_ui;
        rom_pixel_skull_ui_reg <= rom_pixel_skull_ui;
        rom_pixel_id_ui_reg <= rom_pixel_id_ui;
        rom_pixel_output_ui_reg <= rom_pixel_output_ui;
        rom_pixel_output_ui_reg_2 <= rom_pixel_output_ui_2;
        rom_pixel_title_ui_reg <= rom_pixel_title_ui;
        for (i = 0; i < active_zombies; i = i + 1) begin
            if (i < active_zombies && rom_pixel_zombie_ui[i] != 0 &&
                ↪ dead[i]) begin
                rom_pixel_zombie_ui_reg[i][11:8] <= 6; // 2 shades
                ↪ darker than injured
                rom_pixel_zombie_ui_reg[i][7:4] <=
                ↪ rom_pixel_zombie_ui[i][7:4];
                rom_pixel_zombie_ui_reg[i][3:0] <=
                ↪ rom_pixel_zombie_ui[i][3:0];
            end else begin
                rom_pixel_zombie_ui_reg[i] <= rom_pixel_zombie_ui[i];
            end
        end
    end
end

end

// Final pixel selection

```

```

always @* begin
    if (blt_r != 0) begin // red is the colour to show bullets exist
        print_r = blt_r;
        print_g = blt_g;
        print_b = blt_b;
    end else if (rom_pixel_reg != 0 && !display_wave) begin // might be here
        print_r = (zombie_attack_total) ? (rom_pixel_reg[11:8] + 4'b0011) &
            ↪ 4'b1111 : rom_pixel_reg[11:8];
        print_g = rom_pixel_reg[7:4];
        print_b = rom_pixel_reg[3:0];
    end else if (rom_pixel_end_screen_ui_reg != 0 && (game_over ||
        ↪ game_win)) begin
        print_r = rom_pixel_end_screen_ui_reg[11:8];
        print_g = rom_pixel_end_screen_ui_reg[7:4];
        print_b = rom_pixel_end_screen_ui_reg[3:0];
    end else if (rom_pixel_wave_ui_reg != 0 && display_wave && !(game_over
        ↪ || game_win)) begin
        print_r = rom_pixel_wave_ui_reg[11:8];
        print_g = rom_pixel_wave_ui_reg[7:4];
        print_b = rom_pixel_wave_ui_reg[3:0];
    end else if (rom_pixel_ui_reg != 0 && !(game_over || game_win) &&
        ↪ !display_wave) begin // Doesn't matter if black is overridden as
        ↪ background colour is black
        print_r = rom_pixel_ui_reg[11:8];
        print_g = rom_pixel_ui_reg[7:4];
        print_b = rom_pixel_ui_reg[3:0];
    end else if (rom_pixel_health_ui_reg != 0 && !(game_over || game_win)
        ↪ && !display_wave) begin
        print_r = rom_pixel_health_ui_reg[11:8];
        print_g = rom_pixel_health_ui_reg[7:4];
        print_b = rom_pixel_health_ui_reg[3:0];
    end else if (rom_pixel_sound_ui_reg != 0 && !(game_over || game_win) &&
        ↪ !display_wave) begin
        print_r = rom_pixel_sound_ui_reg[11:8];
        print_g = rom_pixel_sound_ui_reg[7:4];
        print_b = rom_pixel_sound_ui_reg[3:0];
    end else if (rom_pixel_title_ui_reg != 0 && !(game_over || game_win) &&
        ↪ !display_wave) begin
        print_r = rom_pixel_title_ui_reg[11:8];
        print_g = rom_pixel_title_ui_reg[7:4];
        print_b = rom_pixel_title_ui_reg[3:0];
    end else if (rom_pixel_skull_ui_reg != 0 && !(game_over || game_win) &&
        ↪ !display_wave) begin
        print_r = rom_pixel_skull_ui_reg[11:8];
        print_g = rom_pixel_skull_ui_reg[7:4];
        print_b = rom_pixel_skull_ui_reg[3:0];
    end else if (rom_pixel_id_ui_reg != 0 && !(game_over || game_win) &&
        ↪ !display_wave) begin
        print_r = rom_pixel_id_ui_reg[11:8];
        print_g = rom_pixel_id_ui_reg[7:4];
        print_b = rom_pixel_id_ui_reg[3:0];
    end
end

```

```

end else if (rom_pixel_output_ui_reg != 0 && !(game_over || game_win)
↳ && !display_wave) begin
    print_r = rom_pixel_output_ui_reg[11:8];
    print_g = rom_pixel_output_ui_reg[7:4];
    print_b = rom_pixel_output_ui_reg[3:0];
end else if (rom_pixel_output_ui_reg_2 != 0 && !(game_over || game_win)
↳ && !display_wave) begin
    print_r = rom_pixel_output_ui_reg_2[11:8];
    print_g = rom_pixel_output_ui_reg_2[7:4];
    print_b = rom_pixel_output_ui_reg_2[3:0];
end else begin
    print_r = 0;
    print_g = 0;
    print_b = 0;

    for (i = 0; i < active_zombies; i = i + 1) begin
        if (rom_pixel_zombie_ui_reg[i] != 0 && !display_wave) begin
            print_r <= (hit_timer[i] > 0 && !dead[i]) ? 4'h8 :
↳ rom_pixel_zombie_ui_reg[i][11:8];
            print_g <= rom_pixel_zombie_ui_reg[i][7:4];
            print_b <= rom_pixel_zombie_ui_reg[i][3:0];
        end
    end

end

end

// END CLOCK DRIVEN LOGIC

// SPRTCLK DRIVEN LOGIC
// Slower to save processing power

// Noise animation calculations

always @(posedge sprtclk or negedge rst) begin
    if (!rst) begin
        sound_state <= 0;
    end else begin
        if (move_animation_true && shoot_true) begin
            sound_state_buffer <= 2; // Sprinting adds 2
        end else if (move_animation_true) begin
            sound_state_buffer <= 1; // Moving adds 1
        end else begin
            sound_state_buffer <= 0;
        end

        if (shoot_true) begin
            case (sw)
                2'd0: sound_state_buffer_gun <= 2; // Assault rifle adds 2
            end
        end
    end
end

```

```

        2'd1: sound_state_buffer_gun <= 1; // Pistol adds 1
        2'd2: sound_state_buffer_gun <= 3; // Shotgun adds 3
        2'd3: sound_state_buffer_gun <= 0; // Knife adds no noise
    endcase
end else begin
    sound_state_buffer_gun <= 0;
end

if (sound_state + sound_state_buffer_gun + pdm_output > 5) begin
    sound_state <= 5; // Cap the value at 5
end else begin
    sound_state <= sound_state_buffer + sound_state_buffer_gun +
        ↪ pdm_output;
end
end
end
end

```

```

// Wave calculations
always @(posedge sprtclk or negedge rst) begin
    if (!rst) begin
        wave_counter <= 0; // Start with the first wave
        wave_state <= 0;
        wave_end_state <= 4;
        display_wave <= 1;

        animation_counter <= 0;
        score <= 0;

        kill_count <= 0;

    end else begin
        score <= score_buffer;

        if (score >= active_zombies && !game_win && active_zombies != 8)
            ↪ begin
                kill_count <= kill_count + score;
                score <= 0;
                if (wave_counter <= 4) begin
                    wave_counter <= (game_over || game_win) ? wave_counter :
                        ↪ wave_counter + 1; // Move to the next wave
                    display_wave <= (game_over || game_win) ? 0:1;
                end
            end
        end

        // Contains the final frame the wave animation should display
        case(wave_counter)
            0: wave_end_state = 0;
            1: wave_end_state = 3;
            2: wave_end_state = 6;

```

```

        3: wave_end_state = 9;
        4: wave_end_state = 12;
        default: wave_end_state = 0;
    endcase

    // Wave changing logic
    if (display_wave) begin
        if (wave_state >= wave_end_state) begin
            if (wave_end_state > 1) begin
                // Updates wave state for the next wave
                wave_state <= wave_end_state; // -3
            end else begin
                wave_state <= 0;
            end
            display_wave <= 0;
        end else begin
            if(animation_counter >= ANIMATION_DELAY) begin
                animation_counter <= 0;
                wave_state <= wave_state + 1;
            end else begin
                animation_counter <= animation_counter + 1;
            end
        end
    end

end

end

// Game Win/Lose
// Would have been easier to displace by a variable instead of storing a
↳ seperate coe file for each animation
always @(posedge sprtclk or negedge rst) begin
    if (!rst) begin
        end_screen_state <= 7;
        game_over <= 0;
        game_win <= 0;
        game_over_counter <= 0;
        end_screen_stage <= 0;
    end else begin

        // Causes the final screen to dis
        if (game_over_counter > 4) begin
            if (end_screen_stage != 3) begin
                end_screen_stage <= end_screen_stage + 1;
            end else begin
                end_screen_stage <= 0;
            end
        end
        if (kill_count + score == 30) begin
            end_screen_state <= 1;
            game_win <= 1;
        end
    end
end

```

```

end
// For displaying game_over or game_win
if (game_over) begin
    case (end_screen_stage)
        0: end_screen_state <= 3;
        1: end_screen_state <= 4;
        2: end_screen_state <= 5;
        3: end_screen_state <= 4;
    endcase
end else if (game_win) begin
    case (end_screen_stage)
        0: end_screen_state <= 0;
        1: end_screen_state <= 1;
        2: end_screen_state <= 2;
        3: end_screen_state <= 1;
    endcase
end
game_over_counter <= 0;
end else begin
    game_over_counter <= game_over_counter + 1;
end

if (health_state == 5) begin
    game_over <= (game_win) ? 0:1;
end
end
end

// Health logic
always @(posedge sprtclk or negedge rst) begin
    if (!rst) begin
        injure_player <= 0;
        health_timer <= 0;
        health_state <= 0;
    end else begin
        if (zombie_attack_total) begin
            injure_player <= 1;
            if(health_timer == 0) begin
                health_state <= health_state + 1;
                health_timer <= 30; // Reset timer for 2.5 seconds
            end
        end else begin
            injure_player <= 0;
        end

        if (display_wave) begin
            health_state <= 0;
        end

        if (health_timer > 0) begin
            health_timer <= health_timer - 1;
        end
    end
end

```

```

        end
    end
end

// Soldier sprite counter
always @(posedge sprtclk or negedge rst) begin
    if (!rst) begin
        sprite_direction <= 0;
        sprite_counter <= 0;
    end else begin
        sprite_direction <= dir;
        if (!move_animation_true || game_over || display_wave || game_win)
            ↪ begin
                sprite_counter <= 0; // Reset the sprite counter on reset or no
                ↪ movement
            end else if (sprite_counter == 4'd11) begin
                sprite_counter <= 0;
            end else begin
                sprite_counter <= sprite_counter + 1; // Increment the counter
            end
        end
    end
end

// END SPRTCLK DRIVEN LOGIC

// GENERAL CALCULATIONS

// Bullet wire to reg
always @* begin
    for (i = 0; i < 10; i = i + 1) begin
        bullet_x_arr[i] = bullet_x[11*i +: 11];
        bullet_y_arr[i] = bullet_y[10*i +: 10];
    end
end

// Creates new zombies each wave, here to allow configuration and clarity
always @* begin
    case (wave_counter)
        0: active_zombies = 4; // Wave 1
        1: active_zombies = 5; // Wave 2
        2: active_zombies = 6; // Wave 3
        3: active_zombies = 7; // Wave 4
        4: active_zombies = 8; // Wave 5
        default: active_zombies = 8; // Safety default
    endcase
end

// Score buffer
always @* begin
    score_buffer_squared = 0;
end

```



```

    for (i = 0; i < active_zombies; i=i+1) begin
        score_buffer_squared = score_buffer_squared + dead[i];
    end
    score_buffer = score_buffer_squared;

end

// Soldier animation assigner (12 frames in 1 cycle, 9 frames stored)
always @* begin
    if (injure_player && health_timer[1]) begin
        sprite_animation_stage = 8;
    end else if (injure_player && !health_timer[1]) begin
        sprite_animation_stage = 9;
    end
    if (move_animation_true) begin //
        case (sprite_counter)
            4'd0: sprite_animation_stage = 0;
            4'd1: sprite_animation_stage = 1;
            4'd2: sprite_animation_stage = 2;
            4'd3: sprite_animation_stage = 3;
            4'd4: sprite_animation_stage = 2;
            4'd5: sprite_animation_stage = 1;
            4'd6: sprite_animation_stage = 0;
            4'd7: sprite_animation_stage = 4;
            4'd8: sprite_animation_stage = 5;
            4'd9: sprite_animation_stage = 6;
            4'd10: sprite_animation_stage = 5;
            4'd11: sprite_animation_stage = 4;
            default: sprite_animation_stage = 0;
        endcase
    end else begin
        sprite_animation_stage = 0;
    end
    if (shoot_true && shoot_flag && !move_animation_true) begin
        sprite_animation_stage = 7;
    end else if (shoot_true) begin
        sprite_animation_stage = 0;
    end
end

end

// END GENERAL CALCULATIONS

assign draw_r = (print_r != 4'b0000 && (rom_pixel_background!=12'h321 ||
↪ rom_pixel_wave_ui!=0 || rom_pixel_end_screen_ui!=0)) ? print_r : bg_r;
assign draw_g = (print_g != 4'b0000 && (rom_pixel_background!=12'h321 ||
↪ rom_pixel_wave_ui!=0 || rom_pixel_end_screen_ui!=0)) ? print_g : bg_g;
assign draw_b = (print_b != 4'b0000 && (rom_pixel_background!=12'h321 ||
↪ rom_pixel_wave_ui!=0 || rom_pixel_end_screen_ui!=0)) ? print_b : bg_b;

// CALLING ANIMATION INSTANCES

```

```

bullet_logic bullet_logic_inst
(
    .clk(clk),
    .rst(rst),
    .sw(sw),
    .blkpos_x(blkpos_x),
    .blkpos_y(blkpos_y),
    .game_clk(game_clk),
    .shoot_true(shoot_true),
    .sprite_direction(sprite_direction),
    .bullet_x(bullet_x),
    .bullet_y(bullet_y),
    .shoot_flag(shoot_flag),
    .move_animation_true(move_animation_true)
);

clk_conv clk_conv_inst
(
    .clk(clk),
    .rst(rst),
    .sprtclk(sprtclk),
    .new_clk(new_clk)
);

Background_Final Background (
    .clka(clk),
    .addra(computed_bg_addr),
    .douta(rom_pixel_background)
);

Soldier_Frames Soldier_Frames (
    .clka(clk),
    .addra(computed_addr),
    .douta(rom_pixel)
);

Weapons_UI Weapons_UI (
    .clka(clk),
    .addra(ui_addr_reg),
    .douta(rom_pixel_ui)
);

Health_UI Health_UI (
    .clka(clk),
    .addra(health_ui_addr_reg),
    .douta(rom_pixel_health_ui)
);

Sound_UI Sound_UI (
    .clka(clk),
    .addra(sound_ui_addr_reg),

```

```

        .douta(rom_pixel_sound_ui)
    );

Wave_UI Wave_UI (
    .clka(clk),
    .addra(wave_ui_addr_reg),
    .douta(rom_pixel_wave_ui)
);

Title Title (
    .clka(clk),
    .addra(title_ui_addr_reg),
    .douta(rom_pixel_title_ui)
);

studentid studentid (
    .clka(clk),
    .addra(id_ui_addr_reg),
    .douta(rom_pixel_id_ui)
);

skull skull (
    .clka(clk),
    .addra(skull_ui_addr_reg),
    .douta(rom_pixel_skull_ui)
);

output_1 output_1 (
    .clka(clk),
    .addra(output_ui_addr_reg),
    .douta(rom_pixel_output_ui),
    .clkb(clk),
    .addrb(output_ui_addr_reg_2),
    .doutb(rom_pixel_output_ui_2)
);

```

```

YOUWIN_plus_GAMEOVER YOUWIN_plus_GAMEOVER (
    .clka(clk),
    .addra(end_screen_ui_addr_reg),
    .douta(rom_pixel_end_screen_ui)
);

```

```

generate
    for (k = 0; k < NUM_ZOMBIES; k = k + 1) begin : zombie_instances
        zombie zombie_inst (
            .rst(rst),
            .game_clk(game_clk),
            .bullet_x(bullet_x),

```

```

        .bullet_y(bullet_y),
        .player_loc_simple_x(player_loc_simple_x),
        .player_loc_simple_y(player_loc_simple_y),
        .detected(detected[k]),
        .curr_x(curr_x),
        .curr_y(curr_y),
        .zombie_computed_reg(zombie_computed_reg[k]),
        .sprtclk(sprtclk),
        .sound_state(sound_state),
        .hit_timer(hit_timer[k]),
        .zombie_attack(zombie_attack[k]),
        .display_wave(display_wave),
        .seed(12'b110101100000 + k),
        .wave_counter(wave_counter),
        .dead(dead[k])
    );
end
endgenerate

generate
    for (j = 0; j < NUM_ZOMBIES; j = j + 2) begin : zombie_render
        Zombie_Animations Zombie_Animations_inst (
            .clka(clk),
            .addra(zombie_computed_reg[j]),
            .douta(rom_pixel_zombie_ui[j]), // Store ROM output for each
            ↪ zombie
            .clkb(clk),
            .addrb(zombie_computed_reg[j+1]),
            .doutb(rom_pixel_zombie_ui[j+1])
        );
    end
endgenerate

endmodule

```

8.3 zombie.v

```

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪ ///
// Company:
// Engineer:
//
// Create Date: 05.12.2024 13:52:09
// Design Name:
// Module Name: zombie
// Project Name:

```

```

// Target Devices:
// Tool Versions:
// Description: Logic to handle zombie movement and attacks
//
// Dependencies: None
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪ ///

module zombie(
    input rst,
    input game_clk,
    input [5:0] player_loc_simple_x,
    input [4:0] player_loc_simple_y,
    output reg detected,
    input [10:0] curr_x,
    input [10:0] curr_y,
    input [109:0] bullet_x, // Checking collision of bullets with the zombie
    input [99:0] bullet_y,
    output reg [12:0] zombie_computed_reg,
    input sprtclk,
    input [2:0] sound_state,
    output reg [3:0] hit_timer,
    output reg zombie_attack,
    input [2:0] display_wave,
    input wire [11:0] seed,
    input [3:0] wave_counter,
    output reg dead
);

reg [1:0] dir;
reg [5:0] current_loc_x;
reg [4:0] current_loc_y;
reg [5:0] abs_diff_x;
reg [4:0] abs_diff_y;
reg [5:0] counter; // game_clk to 1 cps
reg counter_true;
reg movetrue;
reg [10:0] zom_x; // stores the position of the zombies
reg [9:0] zom_y;
reg [4:0] hit_count; // Counts the number of times the zombie has been hit
↪ by a bullet
reg hit_count_detection; // Checks whether the zombie has been defeated
reg z_move_animation_true; // Signal that controls the zombie's movement

wire [11:0] randomiser; // Randomizes the direction they face during respaum
↪ and when

```

```

reg [4:0] circle_radius;
reg [8:0] circle_radius_sq;
reg [12:0] distance_squared;
reg [3:0] sprite_counter;
reg [2:0] sprite_animation_stage; // For displaying the sprite animations

reg [7:0] shot_by_player_counter;

parameter [10:0] START_X = 750;
parameter [9:0] START_Y = 90;

integer i; // Loop variable for bullet collision with zombie
parameter BULLET_RADIUS = 8; // Bullet hitbox size
parameter BLK_SIZE_X = 24;
parameter BLK_SIZE_Y = 24;
parameter PIXEL_SCALE = 1;
parameter AREA = BLK_SIZE_X * BLK_SIZE_Y;
parameter ATTACK_RANGE = 1;

// Player tracker, chooses direction and detects if it can attack the player
always @(posedge game_clk or negedge rst) begin
    if (!rst) begin
        detected <= 0;
        counter <= 0;
        counter_true <= 0;
        zombie_attack <= 0;

    end else begin
        if (!dead) begin
            if (counter == 30) begin
                counter <= 0;
                counter_true <= 1;
            end else begin
                counter <= counter + 1;
                counter_true <= 0;
            end
            // Zombie tracking logic is on a half second clock to make them
            → feel more sluggish, turning on a dime is unrealistic for a
            → zombie and would lead to clustering
            if (counter_true) begin
                // Compute the absolute differences
                abs_diff_x = (player_loc_simple_x > current_loc_x) ?
                    (player_loc_simple_x - current_loc_x) :
                    (current_loc_x - player_loc_simple_x); // was
                → zom_x before (zom_x represents the
                → pixelated position
            end
        end
    end
end

```

```

abs_diff_y = (player_loc_simple_y > current_loc_y) ?
              (player_loc_simple_y - current_loc_y) :
              (current_loc_y - player_loc_simple_y);

// Compute the distance squared
distance_squared = abs_diff_x * abs_diff_x + abs_diff_y *
↳ abs_diff_y;

// Check if the player is within the circle
if (distance_squared <= circle_radius_sq ||
↳ shot_by_player_counter) begin
    detected <= 1;
    movetrue <= 1;
end else begin
    detected <= 0;
    movetrue <= 0;
end

if (distance_squared <= ATTACK_RANGE && !dead) begin //
↳ Switches zombie_attack to 1 when the zombie is within
↳ close proximity
    zombie_attack <= 1;
end else begin
    zombie_attack <= 0;
end

if (!detected) begin
    if (randomiser[0]) begin
        movetrue <= 1;
        dir <= randomiser[2:1];
    end else begin
        movetrue <= 0;
    end
end else begin
    // Decide direction based on which axis is further from
    ↳ the player
    if (abs_diff_x >= abs_diff_y && !dead) begin
        if (player_loc_simple_x > current_loc_x) begin
            dir <= 2'b01; // Move right
        end else begin
            dir <= 2'b11; // Move left
        end
    end
    end else if (!dead) begin
        if (player_loc_simple_y > current_loc_y) begin
            dir <= 2'b10; // Move down
        end else begin
            dir <= 2'b00; // Move up
        end
    end
end
end

```

```

        end
    end
end
end
end

// Zombie movement
// Zombie movement is synced to gameclk

always @(posedge game_clk or negedge rst) begin
    if (!rst) begin
        zom_x <= START_X + (randomiser % 100);
        zom_y <= START_Y + (randomiser % 25);
        z_move_animation_true <= 0;
    end else if (display_wave) begin
        // Has to be repeated since these are reset between waves, repeat
        → prevents ambiguous clock error
        z_move_animation_true <= 0;
    end else begin
        if (!dead) begin
            // Zombies would initialise to
            if (zom_x == 0 || zom_y == 0 || zom_x < 180 || zom_y < 32) begin
                zom_x <= START_X + (randomiser % 100);
                zom_y <= START_Y + (randomiser % 25);
            end
            if (movetrue) begin
                case(dir)
                    2'b00: begin // up
                        if (zom_y > 11'd48 && !(zom_x > 830 && zom_y < 450)
                            → && !(zom_x > 1050 && zom_y < 680) && !(zom_x >
                            → 1200 && zom_y < 850)) begin
                            zom_y <= zom_y -2;
                            z_move_animation_true <= 1;
                        end
                    end
                    2'b10: begin // down
                        if (zom_y < (11'd900 - 11'd80)) begin
                            zom_y <= zom_y + 2;
                            z_move_animation_true <= 1;
                        end
                    end
                    2'b01: begin // right
                        if (zom_x < 11'd1420 && !(zom_x > 830 && zom_y <
                            → 450) && !(zom_x > 1050 && zom_y < 680) &&
                            → !(zom_x > 1200 && zom_y < 850)) begin
                            zom_x <= zom_x + 2;
                            z_move_animation_true <= 1;
                        end
                    end
                    2'b11: begin // left

```



```

        if (zom_x > 192) begin
            zom_x <= zom_x - 2;
            z_move_animation_true <= 1;
        end
    end
    default: begin
        z_move_animation_true <= 0;
    end
endcase
end else begin
    z_move_animation_true <= 0;
end
end else begin
    z_move_animation_true <= 0;
end
end
end

// Bullet detection logic
always @(posedge game_clk or negedge rst) begin
    if (!rst) begin
        hit_count <= 0;
        hit_count_detection <= 0;
        hit_timer <= 0;
        dead <= 0;
    end else if (display_wave) begin
        dead <= 0;
        hit_count <= 0;
    end else begin
        if (hit_timer > 0) begin
            hit_timer <= hit_timer - 1;
        end
        if (shot_by_player_counter) begin
            hit_count_detection <= 0;
        end
        // Check for bullet collisions
        for (i = 0; i < 10; i = i + 1) begin
            if ((bullet_x[11*i +: 11] >= zom_x - BULLET_RADIUS) &&
                (bullet_x[11*i +: 11] <= zom_x + BLK_SIZE_X +
                 ↪ BULLET_RADIUS) &&
                (bullet_y[10*i +: 10] >= zom_y - BULLET_RADIUS) &&
                (bullet_y[10*i +: 10] <= zom_y + BLK_SIZE_Y +
                 ↪ BULLET_RADIUS)) begin
                hit_count <= (dead) ? hit_count:hit_count + 1; // Increment
                ↪ hit count
                hit_count_detection <= 1;
                hit_timer <= 5'd15; // Highlight the zombie for 15 frames
                dead <= (hit_count > 3 + wave_counter/2); // Freezes the
                ↪ zombie if hit
            end
        end
    end
end
end

```

```

    end
end

// Zombie tracker updater
// This allows the zombies to travel parallel to the player, making them
↪ more difficult to shoot
// This sometimes results in the zombies pacing next to the player, however
↪ they still damage the player so the player should be too distracted to
↪ notice
// Having them stop next to the player didn't feel right
always @(posedge game_clk or negedge rst) begin
    if (!rst) begin
        current_loc_x <= zom_x / 32;
        current_loc_y <= zom_y / 32;
    end else begin
        current_loc_x <= zom_x / 32;
        current_loc_y <= zom_y / 32;
    end
end
end

// Zombie address calculator (with rotation)
always @* begin
    zombie_computed_reg = 0;
    // Determine address based on sprite direction and position
    if (dir == 2'd0 || dir == 2'd2) begin
        if (zom_x <= curr_x && curr_x <= zom_x + BLK_SIZE_X * PIXEL_SCALE -
↪ 1 &&
            zom_y <= curr_y && curr_y <= zom_y + BLK_SIZE_Y * PIXEL_SCALE -
↪ 1) begin

            case (dir)
                2'd2: // Down
                    zombie_computed_reg = sprite_animation_stage * AREA
                        + ((curr_y - zom_y)/PIXEL_SCALE) *
↪ BLK_SIZE_X
                        + ((curr_x - zom_x)/PIXEL_SCALE);

                2'd0: // Up
                    zombie_computed_reg = sprite_animation_stage * AREA
                        + (AREA - 1)
                        - (((curr_y - zom_y)/PIXEL_SCALE) *
↪ BLK_SIZE_X
                        + ((curr_x - zom_x)/PIXEL_SCALE));

            endcase
        end
    end else if (dir == 2'd1 || dir == 2'd3) begin
        if (zom_x <= curr_x && curr_x <= zom_x + BLK_SIZE_Y * PIXEL_SCALE -
↪ 1 &&
            zom_y <= curr_y && curr_y <= zom_y + BLK_SIZE_X * PIXEL_SCALE -
↪ 1) begin

            case (dir)

```

```

2'd3: // Left
    zombie_computed_reg = sprite_animation_stage * AREA +
        ↪ (AREA - 1)
        ↪ - ((curr_x - zom_x)/PIXEL_SCALE) *
        ↪ BLK_SIZE_X
        ↪ + ((curr_y - zom_y)/PIXEL_SCALE);

2'd1: // Right
    zombie_computed_reg = sprite_animation_stage * AREA
        ↪ + ((curr_x - zom_x)/PIXEL_SCALE) *
        ↪ BLK_SIZE_X
        ↪ + ((BLK_SIZE_X - 1 - (curr_y -
        ↪ zom_y)/PIXEL_SCALE));

    endcase
end
end
end

// Zombie animation calculator
always @(posedge sprtclk or negedge rst) begin
    if (!rst) begin
        sprite_counter <= 0;
    end
    else begin
        if (!z_move_animation_true) begin
            sprite_counter <= 0; // Reset the sprite counter on reset or no
            ↪ movement
        end
        else if (sprite_counter == 4'd11) begin
            sprite_counter <= 0;
        end
        else begin
            sprite_counter <= sprite_counter + 1; // Increment the counter
        end
    end
end

// Zombie animation assigner (12 frames in 1 cycle, 7 frames stored)
always @* begin
    if (z_move_animation_true) begin
        case (sprite_counter)
            4'd0: sprite_animation_stage = 0;
            4'd1: sprite_animation_stage = 1;
            4'd2: sprite_animation_stage = 2;
            4'd3: sprite_animation_stage = 3;
            4'd4: sprite_animation_stage = 2;
            4'd5: sprite_animation_stage = 1;
            4'd6: sprite_animation_stage = 0;
            4'd7: sprite_animation_stage = 4;
            4'd8: sprite_animation_stage = 5;
            4'd9: sprite_animation_stage = 6;
            4'd10: sprite_animation_stage = 5;
            4'd11: sprite_animation_stage = 4;

```

```

        default: sprite_animation_stage = 0;
    endcase

    end else begin
        sprite_animation_stage = 0;
    end
end

// Detection rate assignments
always @* begin
    circle_radius = (sound_state + 1) * 4;
    circle_radius_sq = circle_radius * circle_radius;
end

// Creates a timer that waits ~4 seconds, this is used to have the zombie
↳ chase the player after being shot
always @(posedge game_clk or negedge rst) begin
    if (!rst || display_wave) begin
        shot_by_player_counter <= 0;
    end else if (hit_count_detection) begin
        shot_by_player_counter <= 255;
    end else if (shot_by_player_counter != 0) begin
        shot_by_player_counter <= shot_by_player_counter - 1;
    end
end

endmodule

random_number random_number_bullet_inst(
    .clk(game_clk),
    .rst(rst),
    .lfsr(randomiser),
    .seed(seed)
);

```

8.4 bullet_logic.v

```

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↳ ///
// Company:
// Engineer:
//
// Create Date: 05.12.2024 13:52:09
// Design Name:

```

```

// Module Name: bullet_logic
// Project Name:
// Target Devices:
// Tool Versions:
// Description: Logic to handle bullet movement and initialization.
//
// Dependencies: None
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪ ///

module bullet_logic(
    input clk,
    input rst,
    input [1:0] sw,
    input [10:0] blkpos_x,      // Block position X (initial bullet position)
    input [9:0] blkpos_y,      // Block position Y (initial bullet position)
    input game_clk,
    input shoot_true,          // Signal indicating a shot has been fired
    input [1:0] sprite_direction, // Direction of the sprite

    output reg [109:0] bullet_x, // Bullet positions (X-coordinates) as a
    ↪ 110-bit wire
    output reg [99:0] bullet_y,  // Bullet positions (Y-coordinates) as a
    ↪ 100-bit wire
    output reg shoot_flag,

    input move_animation_true
);

    reg [11:0] randomiser;
    wire [11:0] random_capture;

    // Internal arrays for storing bullet positions and velocities
    reg [10:0] bullet_x_array [0:9]; // Array for X positions of the 10 bullets
    reg [9:0] bullet_y_array [0:9];  // Array for Y positions of the 10 bullets
    reg active_flag [0:9];
    reg signed [9:0] bullet_velocity_x [0:9]; // Velocity in the X direction
    reg signed [9:0] bullet_velocity_y [0:9]; // Velocity in the Y direction
    reg loop;
    integer i; // Loop variable

    reg [3:0] next_bullet; // Points to the next bullet to activate
    reg signed [11:0] bullet_x_array_signed [0:9];
    reg signed [10:0] bullet_y_array_signed [0:9];
    reg [8:0] counter;

```

```

reg [2:0] spread; // Spreads the bullets across the shooting direction
reg [3:0] shotgun_bullet_counter; // Counts the number of bullets being
↳ outputed from the shotgun
reg [5:0] bullet_lifetime_other [0:9]; // Used for stopping the bullet'sm
↳ after a certain range

reg [6:0] bullet_range_var;
reg signed [5:0] bullet_incrementer;

parameter PIXEL_SCALE = 1;
parameter SHOTGUN_LIFETIME = 6; // Reduces the range of the SHOTGUN
parameter FOR_LOOP_ENDER = 1;

// Bullet movement and reset logic
always @(posedge game_clk or negedge rst) begin
    if (!rst) begin
        // Reset all bullets and velocities
        for (i = 0; i < 10; i = i + 1) begin
            bullet_x_array_signed[i] <= 11'b11111111111;
            bullet_y_array_signed[i] <= 10'b11111111111;
            active_flag[i] <= 0;
            bullet_velocity_x[i] <= 6'd0;
            bullet_velocity_y[i] <= 6'd0;

        end
        next_bullet <= 4'd0; // Reset pointer
        shotgun_bullet_counter <= 0;
        shoot_flag <= 0;
        bullet_incrementer <= 0;
    end else begin
        // Bullet movement logic
        for (i = 0; i < 10; i = i + 1) begin
            if (active_flag[i]) begin

                if ((bullet_x_array_signed[i] > 850 &&
↳ bullet_y_array_signed[i] < 450) ||
                    (bullet_x_array_signed[i] > 1000 &&
↳ bullet_y_array_signed[i] < 650) ||
                    (bullet_x_array_signed[i] > 1175 &&
↳ bullet_y_array_signed[i] < 830
                    || bullet_x_array_signed[i] < 190)) begin

                    active_flag[i] <= 0;
                end
                // Update position based on velocity
                bullet_x_array_signed[i] <= bullet_x_array_signed[i] +
↳ bullet_velocity_x[i];
                bullet_y_array_signed[i] <= bullet_y_array_signed[i] +
↳ bullet_velocity_y[i];
            end
        end
    end
end

```

```

bullet_lifetime_other[i] <= bullet_lifetime_other[i] +
    ↪ 1;
case (sw)
    2'd0: bullet_range_var <= 10; // AR
    2'd1: bullet_range_var <= 15; // Pistol
    2'd2: bullet_range_var <= 5; // Shotgun
    2'd3: bullet_range_var <= 63; // Knife

endcase
if (active_flag[i]) begin
    if (bullet_lifetime_other[i] >= bullet_range_var ||
        bullet_x_array_signed[i] > 11'd1429 ||
        ↪ bullet_y_array_signed[i] > 10'd889 ||
        bullet_x_array_signed[i] < 10'd0 ||
        ↪ bullet_y_array_signed[i] < 10'd0) begin
        active_flag[i] <= 0;
    end
end

// Deactivate bullet if it moves off-screen
if (bullet_x_array_signed[i] > 11'd1429 ||
    ↪ bullet_y_array_signed[i] > 10'd889 ||
    ↪ bullet_x_array_signed[i] < 10'd0 ||
    ↪ bullet_y_array_signed[i] < 10'd0) begin
    active_flag[i] <= 0; // Bullet is off-screen, deactivate
    ↪ it
end
end
end
case (sw)
    2'd1: begin // Pistol
        spread <= 2;
        if (!shoot_true) begin
            shoot_flag <= 1;
        end
    end
    2'd2: begin // shotgun
        spread <= 4;
        if (!shoot_true) begin
            shoot_flag <= 1;
        end
        if (counter == 100) begin
            counter <= 0;
            shoot_flag <= 1;
        end else begin
            counter <= counter + 1;
        end
    end
    2'd3: begin // Knife
        spread <= 1;
        if (!shoot_true) begin

```

```

        shoot_flag <= 1;
    end
    if (counter == 100) begin
        counter <= 0;
        shoot_flag <= 1;
    end else begin
        counter <= counter + 1;
    end
end
default: begin // AR
    spread <= 2;
    if (counter == 5) begin
        counter <= 0;
        shoot_flag <= 1;
    end else begin
        counter <= counter + 1;
    end
end
endcase

if (shoot_true && !active_flag[next_bullet] &&
    ↪ !move_animation_true) begin // When sprite is moving it is used
    ↪ as a speed boost

    if (shoot_flag) begin
        if (sw == 2'd2) begin // Shotgun selected
            // Initialize the next bullet
            bullet_x_array_signed[next_bullet] <= blkpos_x;
            bullet_y_array_signed[next_bullet] <= blkpos_y;
            active_flag[next_bullet] <= 1;
            bullet_lifetime_other[next_bullet] <= 0; // Reset
            ↪ lifetime for shotgun bullets

            // Assign velocity based on `shotgun_bullet_counter`
            case (sprite_direction)
                2'd0: begin // Up
                    bullet_x_array_signed[next_bullet] <= blkpos_x
                    ↪ + 16 * PIXEL_SCALE;
                    bullet_y_array_signed[next_bullet] <= blkpos_y
                    ↪ + 33 * PIXEL_SCALE;
                    bullet_incrementer <= randomiser[2:0] +
                    ↪ next_bullet[0];
                    bullet_velocity_x[next_bullet] <=
                    ↪ (bullet_incrementer - 4) * 2;
                    bullet_velocity_y[next_bullet] <= 10'd40; //
                    ↪ Consistent upward velocity
                end
                2'd1: begin // Right
                    bullet_y_array_signed[next_bullet] <= blkpos_y
                    ↪ + 12 * PIXEL_SCALE;

```



```

        bullet_x_array_signed[next_bullet] <= blkpos_x
        ↪ + 33 * PIXEL_SCALE;
        bullet_incrementer <= randomiser[5:3] +
        ↪ next_bullet[0];
        bullet_velocity_x[next_bullet] <= 10'd40; //
        ↪ Consistent rightward velocity
        bullet_velocity_y[next_bullet] <=
        ↪ (bullet_incrementer - 4) * 2;
    end
2'd2: begin // Down
        bullet_x_array_signed[next_bullet] <= blkpos_x
        ↪ + 16 * PIXEL_SCALE;
        bullet_incrementer <= randomiser[8:6] +
        ↪ next_bullet[0];
        bullet_velocity_x[next_bullet] <=
        ↪ (bullet_incrementer - 4) * 2;
        bullet_velocity_y[next_bullet] <= -10'd40; //
        ↪ Consistent downward velocity
    end
2'd3: begin // Left
        bullet_y_array_signed[next_bullet] <= blkpos_y
        ↪ + 5 * PIXEL_SCALE;
        bullet_velocity_x[next_bullet] <= -10'd40; //
        ↪ Consistent leftward velocity
        bullet_incrementer <= randomiser[11:9] +
        ↪ next_bullet[0];
        bullet_velocity_y[next_bullet] <=
        ↪ (bullet_incrementer - 4) * 2;
    end
endcase

// Move pointer to the next bullet slot
next_bullet <= (next_bullet + 4'd1) % 4'd10;
shotgun_bullet_counter <= shotgun_bullet_counter + 1;
// Reset after 3 bullets are fired
if (shotgun_bullet_counter == 3) begin
    shotgun_bullet_counter <= 0; // Reset counter
    shoot_flag <= 0; // Reset shoot flag
end
end
else begin
    // Other weapons
    for (i = 0; i < FOR_LOOP_ENDER; i = i + 1) begin

        // Initialize the next bullet
        bullet_x_array_signed[next_bullet] <= blkpos_x;
        bullet_y_array_signed[next_bullet] <= blkpos_y;
        active_flag[next_bullet] <= 1;
        bullet_lifetime_other[next_bullet] <= 0;
    end
end

```

```

// Set velocity based on sprite direction
case (sprite_direction)
  2'd0: begin // Up
    bullet_x_array_signed[next_bullet] <=
      ↪ blkpos_x + 16 * PIXEL_SCALE;
    bullet_y_array_signed[next_bullet] <=
      ↪ blkpos_y + 33 * PIXEL_SCALE;
    bullet_incrementer <= randomiser[2:0] +
      ↪ next_bullet[0];
    bullet_velocity_x[next_bullet] <= (spread
      ↪ != 1) ? (bullet_incrementer - 4) *
      ↪ spread / 2 : 0; // This halves the
      ↪ chance of extreme -4 and 4 values
    bullet_velocity_y[next_bullet] <= 10'd40;
  end
  2'd1: begin // Right
    bullet_y_array_signed[next_bullet] <=
      ↪ blkpos_y + 12 * PIXEL_SCALE;
    bullet_x_array_signed[next_bullet] <=
      ↪ blkpos_x + 33 * PIXEL_SCALE;
    bullet_incrementer <= randomiser[5:3] +
      ↪ next_bullet[0];
    bullet_velocity_x[next_bullet] <= 10'd40;
    bullet_velocity_y[next_bullet] <= (spread
      ↪ != 1) ? (bullet_incrementer - 4) *
      ↪ spread / 2 : 0;
  end
  2'd2: begin // Down
    bullet_x_array_signed[next_bullet] <=
      ↪ blkpos_x + 16 * PIXEL_SCALE;
    bullet_incrementer <= randomiser[8:6] +
      ↪ next_bullet[0];
    bullet_velocity_x[next_bullet] <= (spread
      ↪ != 1) ? (bullet_incrementer - 4) *
      ↪ spread / 2 : 0;
    bullet_velocity_y[next_bullet] <= - 10'd40;
  end
  2'd3: begin // Left
    bullet_y_array_signed[next_bullet] <=
      ↪ blkpos_y + 5 * PIXEL_SCALE;
    bullet_incrementer <= randomiser[11:9] +
      ↪ next_bullet[0];
    bullet_velocity_x[next_bullet] <= - 10'd40;
    bullet_velocity_y[next_bullet] <= (spread
      ↪ != 1) ? (bullet_incrementer - 4) *
      ↪ spread / 2 : 0;
  end
endcase

// Move pointer to the next bullet slot
next_bullet <= (next_bullet + 4'd1) % 4'd10;

```

```

        shoot_flag <= 0;
    end
end
end
end
end

// Update bullet output registers
always @* begin
    for (i = 0; i < 10; i = i + 1) begin
        if (active_flag[i]) begin
            bullet_x_array[i] = bullet_x_array_signed[i];
            bullet_y_array[i] = bullet_y_array_signed[i];
            bullet_x[11*i +: 11] = bullet_x_array[i];
            bullet_y[10*i +: 10] = bullet_y_array[i];
        end else begin
            bullet_x[11*i +: 11] = 11'b11111111111;
            bullet_y[10*i +: 10] = 10'b1111111111;
        end
    end
end

// Randomises bullet values
always @(posedge clk) begin
    randomiser <= random_capture; // Keep sampling the LFSR every clock
end

random_number random_number_bullet_inst(
    .clk(clk),
    .rst(rst),
    .lfsr(random_capture),
    .seed(12'b101001110100)
);

endmodule

```

8.5 pdm_mic_interface.v

```

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪ // Company:
// Engineer:
//
// Create Date: 05.01.2025 19:37:54
// Design Name:
// Module Name: pdm_mic_interface
// Project Name:
// Target Devices:

```

```

// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////
↪  //////////

module pdm_mic_interface(
    input wire clk,          // System clock (e.g., 100 MHz)
    input wire rst,          // Reset signal
    input wire pdm_data,     // PDM data input
    output wire pdm_clk,     // PDM clock output
    output reg [1:0] pdm_output, //Output for sound level classification
    output reg [6:0] seg,    // 7 segment display
    input wire game_clk_three
);

// Parameters
parameter PDM_CLK_DIV = 40; // Divider for PDM clock (e.g., 100 MHz / 40
↪  = 2.5 MHz)
parameter DECIMATION_FACTOR = 64; // Number of PDM samples per PCM
↪  sample

parameter AMBIENT = 2;
parameter CONVERSATIONAL = 3;
parameter LOUD = 4;

// Clock divider for PDM clock
reg [5:0] clk_div_counter = 0;
reg pdm_clk_reg = 0;
reg [15:0] pcm_data; // Decimated PCM output
reg [3:0] game_clk_three_div;
reg second_clk;
reg [15:0] max_value;
reg [1:0] max_rst_count;

// Generates a slower clock signal second_clk which runs at ~ 3Hz
always @(posedge game_clk_three or negedge rst) begin
    if (!rst) begin
        game_clk_three_div <= 0;
        second_clk <= 0;
    end else begin
        if (game_clk_three_div >= 10) begin
            second_clk <= ~second_clk;
            game_clk_three_div <= 0;
        end
    end
end

```

```

        end else begin
            game_clk_three_div <= game_clk_three_div + 1;
        end
    end
end

// Updates pcm_data with the current max_value
always @(posedge second_clk or negedge rst) begin
    if (!rst) begin
        pcm_data <= 0;
        max_rst_count <= 0;
    end else begin
        if (max_value > 0) begin
            pcm_data <= max_value;
        end
        if (max_rst_count >= 3) begin
            max_rst_count <= 0;
        end else begin
            max_rst_count <= max_rst_count + 1;
        end
    end
end

// Generates a PDM clock signal for decimating PDM data
always @(posedge clk or negedge rst) begin
    if (!rst) begin
        clk_div_counter <= 0;
        pdm_clk_reg <= 0;
    end else begin
        if (clk_div_counter == (PDM_CLK_DIV - 1)) begin
            clk_div_counter <= 0;
            pdm_clk_reg <= ~pdm_clk_reg;
        end else begin
            clk_div_counter <= clk_div_counter + 1;
        end
    end
end

assign pdm_clk = pdm_clk_reg;

// Decimation counter and accumulator
reg [5:0] decimation_counter = 0;
reg [15:0] accumulator = 0;

// Performs decimation of PDM data to produce PCM data and tracks
→ maximum PCM value
always @(posedge pdm_clk_reg or negedge rst) begin
    if (!rst) begin
        decimation_counter <= 0;
        accumulator <= 0;
    end
end

```

```

        max_value <= 0;
    end else
        if (max_rst_count >= 3) begin // every 3 seconds
            max_value <= 0;
        end else begin
            if (decimation_counter == (DECIMATION_FACTOR - 1)) begin
                decimation_counter <= 0;
                if (max_value < accumulator) begin
                    max_value <= accumulator;
                end
                accumulator <= 0;
            end else begin
                decimation_counter <= decimation_counter + 1;
                accumulator <= accumulator + pdm_data;
            end
        end
    end
end

// 7-Segment Decoder
always @(pcm_data) begin
    case (pcm_data[3:0]) // Display lower nibble of pcm_data
        4'h0: seg = 7'b0000001; // 0
        4'h1: seg = 7'b1001111; // 1
        4'h2: seg = 7'b0010010; // 2
        4'h3: seg = 7'b0000110; // 3
        4'h4: seg = 7'b1001100; // 4
        4'h5: seg = 7'b0100100; // 5
        4'h6: seg = 7'b0100000; // 6
        4'h7: seg = 7'b0001111; // 7
        4'h8: seg = 7'b0000000; // 8
        4'h9: seg = 7'b0000100; // 9
        4'hA: seg = 7'b0000100; // A
        4'hB: seg = 7'b1100000; // B
        4'hC: seg = 7'b0110001; // C
        4'hD: seg = 7'b1000010; // D
        4'hE: seg = 7'b0010000; // E
        4'hF: seg = 7'b0111000; // F
        default: seg = 7'b0000000; // Blank
    endcase
end

// Assigning different variations in sound of pcm_data to pdm_output
always @(pcm_data) begin
    if (pcm_data[3:0] > LOUD) pdm_output <= 2'b11;
    else if (pcm_data[3:0] > CONVERSATIONAL) pdm_output <= 2'b10;
    else if (pcm_data[3:0] > AMBIENT) pdm_output <= 2'b01;
    else pdm_output <= 2'b00;
end

endmodule

```

8.6 clk_conv.v

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
↪ ///
// Company:
// Engineer:
//
// Create Date: 06.12.2024 13:53:37
// Design Name:
// Module Name: clk_conv
// Project Name:
// Target Devices:
// Tool Versions:
// Description: Clock conversion module to generate `sprtclk`
//              based on counters from the input clock `clk`.
//
// Dependencies: None
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
↪ ///

// Generates a sprtclk for processing sprite animations

module clk_conv(
    input clk,           // Input clock signal
    input rst,           // Active-low reset signal
    output reg sprtclk,   // Output clock for sprite logic
    output reg new_clk
);

    reg [26:0] counterclk; // Counter for clock division
    reg [3:0] sprtclk_counter;

    always @(posedge clk or negedge rst) begin
        if (!rst) begin
            counterclk <= 27'd0; // Reset counter to 0
            sprtclk <= 1'b0;      // Reset `sprtclk`
            sprtclk_counter <= 4'd0;
            new_clk <= 1'b0;
        end else begin
            // Increment the counter
            counterclk <= counterclk + 1;

            // Generate sprtclk every 5 million cycles and reset the counter
            if (counterclk >= 27'd5000000) begin
                sprtclk <= ~sprtclk;
                counterclk <= 27'd0; // Reset the counter
            end
        end
    end
endmodule
```

```

        end

        sprtclk_counter <= sprtclk_counter + 1;
        if(sprtclk_counter == 4'd9) begin
            new_clk <= ~new_clk;
            sprtclk_counter <= 4'd0;
        end
    end
end
endmodule

```

8.7 vga.v

```

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪ ///
// Company:
// Engineer:
//
// Create Date: 21.10.2022 13:35:16
// Design Name:
// Module Name: vga
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪ ///

module vga(
    input clk,
    input rst,
    // input [2:0] sw,
    input [3:0] draw_r,
    input [3:0] draw_g,
    input [3:0] draw_b,
    output [10:0] curr_x,
    output [10:0] curr_y,
    output [3:0] pix_r,
    output [3:0] pix_g,
    output [3:0] pix_b,

```



```

    output hsync,
    output vsync
);

//internal signals
reg [10:0] hcount;
reg [9:0] vcount;
reg [10:0] curr_x_r;
reg [10:0] curr_y_r;

wire line_end = (hcount == 11'd1903);
wire frame_end = (vcount == 10'd931);
//wire pixclk;

//hsync vsync assing combinational
assign hsync = ((hcount >= 11'd0) && (hcount <= 11'd151));
assign vsync = ((vcount >= 10'd0) && (vcount <= 10'd2));
assign display_region = (hcount >= 11'd384) && (hcount <= 11'd1823) && (vcount
↳ >= 10'd31) && (vcount <= 10'd930);

//pix assign combinational
assign pix_r = (display_region) ? draw_r : 4'b0000;
assign pix_g = (display_region) ? draw_g : 4'b0000;
assign pix_b = (display_region) ? draw_b : 4'b0000;

//hcount synchronous
always@(posedge clk) begin
    if (!rst)
        hcount <= 11'd0;
    else begin
        if(line_end)
            hcount <= 11'd0;
        else
            hcount <= hcount + 11'd1;
    end
end

//vcount synchronous
always@(posedge clk) begin
    if (!rst)
        vcount <= 10'd0;
    else begin
        if(frame_end) begin
            vcount <= 10'd0;
        end else begin
            if(line_end) begin
                vcount <= vcount + 1;
            end
        end
    end
end

```



```

//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////
↪ ///

// Generates a pseudo random 8 bit number

module random_number (
    input wire      clk,
    input wire      rst,
    output reg [11:0] lfsr,    // 12-bit LFSR
    input wire [11:0] seed
);
    wire feedback;

    assign feedback = lfsr[11] ^ lfsr[10] ^ lfsr[9] ^ lfsr[3];

    always @(posedge clk or negedge rst) begin
        if (!rst) begin
            // Load the seed during reset
            lfsr <= seed;
        end else begin
            // Shift left and insert feedback in the LSB
            lfsr <= {lfsr[10:0], feedback};
        end
    end
endmodule

```

8.9 bullet_test.v

```

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////
↪ ///
// Company:
// Engineer:
//
// Create Date: 09.01.2025 17:03:18
// Design Name:
// Module Name: bullet_test
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:

```

```

// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
↪   ///

module bullet_test;

    reg clk;
    reg rst;
    reg [1:0] sw;
    reg [10:0] blkpos_x;      // Block position X (initial bullet position)
    reg [9:0] blkpos_y;      // Block position Y (initial bullet position)
    reg game_clk;
    reg shoot_true;          // Signal indicating a shot has been fired
    reg [1:0] sprite_direction; // Direction of the sprite
    reg move_animation_true;

    integer i;

    wire [109:0] bullet_x;
    wire [99:0] bullet_y;
    wire shoot_flag;

    bullet_logic uut(
        .clk(clk),
        .rst(rst),
        .sw(sw),
        .blkpos_x(blkpos_x),      // Block position X (initial bullet position)
        .blkpos_y(blkpos_y),      // Block position Y (initial bullet
        ↪ position)
        .game_clk(game_clk),
        .shoot_true(shoot_true),    // Signal indicating a shot has been
        ↪ fired
        .sprite_direction(sprite_direction), // Direction of the sprite

        .bullet_x(bullet_x), // Bullet positions (X-coordinates) as a 110-bit
        ↪ wire
        .bullet_y(bullet_y), // Bullet positions (Y-coordinates) as a 100-bit
        ↪ wire
        .shoot_flag(shoot_flag),

        .move_animation_true(move_animation_true)
    );
    // Clock generation for game_clk
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // 100MHz clock
    end
end

```

```

// Clock generation for game_clk
initial begin
    game_clk = 0;
    forever #10 game_clk = ~game_clk; // 50MHz clock
end

initial begin
    rst = 1;
    blkpos_x = 500;
    blkpos_y = 560;
    shoot_true = 1;

    sprite_direction = 2'd1; // Right direction assigned initially
    move_animation_true = 0;
    sw = 2'd00; // Default weapon (AR)

    // Apply reset
    #20 rst = 0;
    #20 rst = 1;

    // Test Case 1: Shooting AR
    sw = 2'b00;
    for (i = 0; i < 5; i = i + 1) begin
        shoot_true = 1;
        #25;
        shoot_true = 0;
        #25;
    end

    shoot_true = 1;
    #250
    shoot_true = 0;

    // Test Case 2: Shooting Pistol
    sw = 2'b01;
    for (i = 0; i < 5; i = i + 1) begin
        shoot_true = 1;
        #25;
        shoot_true = 0;
        #25;
    end

    shoot_true = 1;
    #250
    shoot_true = 0;

    // Test Case 3: Shooting Shotgun
    sw = 2'b10;
    for (i = 0; i < 5; i = i + 1) begin

```

```

        shoot_true = 1;
        #25;
        shoot_true = 0;
        #25;
    end

    shoot_true = 1;
    #250
    shoot_true = 0;

    // Test Case 1: Throwing Knife
    sw = 2'b11;
    for (i = 0; i < 5; i = i + 1) begin
        shoot_true = 1;
        #25;
        shoot_true = 0;
        #25;
    end

    shoot_true = 1;
    #250
    shoot_true = 0;

    // Finish simulation
    #200;
    $stop;
end

initial begin
    $monitor("Time: %d, Direction: %b, Bullet_X: %h, Bullet_Y: %h", $time,
        ↪ sprite_direction, bullet_x, bullet_y);
end

endmodule

```

8.10 sound_tb.v

```

`timescale 1ns / 1ps

module sound_tb;

    // Inputs
    reg rst;
    reg game_clk;
    reg [109:0] bullet_x;
    reg [99:0] bullet_y;
    reg [5:0] player_loc_simple_x; // 6-bit, can go up to 63
    reg [4:0] player_loc_simple_y; // 5-bit

```

```

reg [10:0] curr_x;
reg [10:0] curr_y;
reg [2:0] sound_state;
reg [2:0] display_wave;
reg [3:0] wave_counter;
reg [11:0] seed;

// We'll use integer for loop indices
integer s;
integer i;

// Outputs
wire detected;
wire zombie_attack;
wire [3:0] hit_timer;
wire [12:0] zombie_computed_reg;
wire dead;

// Instantiate the Unit Under Test (UUT)
zombie uut (
    .rst(rst),
    .game_clk(game_clk),
    .bullet_x(bullet_x),
    .bullet_y(bullet_y),
    .player_loc_simple_x(player_loc_simple_x),
    .player_loc_simple_y(player_loc_simple_y),
    .detected(detected),
    .curr_x(curr_x),
    .curr_y(curr_y),
    .sound_state(sound_state),
    .display_wave(display_wave),
    .wave_counter(wave_counter),
    .zombie_computed_reg(zombie_computed_reg),
    .hit_timer(hit_timer),
    .zombie_attack(zombie_attack),
    .seed(seed),
    .dead(dead)
);

// Clock generation for game_clk
initial begin
    game_clk = 0;
    forever #10 game_clk = ~game_clk; // 50MHz clock
end

// Test stimulus
initial begin
    // Initialize
    rst = 1;
    bullet_x = 110'd0;
    bullet_y = 100'd0;

```

```

player_loc_simple_x = 6'd0;    // Consistent with 6-bit
player_loc_simple_y = 5'd3;    // 5-bit
curr_x = 11'd0;
curr_y = 11'd0;
sound_state = 3'd0;
display_wave = 3'd0;
wave_counter = 4'd0;
seed = 12'hA55;    // Arbitrary seed

// Apply reset
#20 rst = 0;
#20 rst = 1;

// Loop over sound_state = 1..5
for (s = 1; s <= 4; s = s + 1) begin
    // Set the new sound_state
    sound_state = s[2:0];    // ensures it fits in 3 bits

    // Let the simulation settle
    #200;
    $display("Testing sound_state = %0d", s);

    // Move player step by step, up to 40 increments
    // waiting for detected == 1
    for (i = 0; i < 40; i = i + 1) begin
        player_loc_simple_x = player_loc_simple_x + 1;
        #300; // 300 ns wait (to allow counter to flip and update abs
            ↪ values)

        // Check if detection occurred
        if (detected == 1'b1) begin
            $display("sound_state=%0d: detected=1 at i=%0d, resetting
                ↪ player/zombie positions", s, i);

            // Example: reset positions

            #900; // Allow abs values to be updated 3 times and show
                ↪ that the zombie is closing in on the player
            // Re-apply reset to the UUT if desired
            #20 rst = 0;
            #20 rst = 1;
            player_loc_simple_x = 6'd0;

            // Let signals settle
            #50;
            // Move on to next sound_state
            i <= 40;
        end
    end
end
end
end

```



```

        // Finish simulation
        #200;
        $finish;
    end

endmodule

```

8.11 coe_to_image_convertor

```

% COE Superimposition Script with Debugging
% File paths
map_coe_file = 'New_Background.coe'; % Map COE file (radix=16)
collision_coe_file = 'collision_data_cropped.coe'; % Collision COE file
↳ (radix=2)
output_image = 'map_with_collision_overlay_1.png'; % Output file

% Step 1: Load Map COE File
fprintf('Loading Map COE file: %s\n', map_coe_file);
map_data = load_coe_file(map_coe_file, 16);

% Define map dimensions
map_width = 720; % Map width in pixels
map_height = length(map_data) / map_width;

if mod(length(map_data), map_width) ~= 0
    error('Map data dimensions do not align with specified map width.');
```

```

end

fprintf('Map Dimensions: %dx%d\n', map_width, map_height);

% Reshape map data
map_image = reshape(map_data, [map_width, map_height]);
map_image_normalized = uint8(255 * mat2gray(map_image)); % Normalize for
↳ grayscale

% Convert map to 3-channel RGB
map_image_rgb = cat(3, map_image_normalized, map_image_normalized,
↳ map_image_normalized);

% Step 2: Load Collision COE File
fprintf('Loading Collision COE file: %s\n', collision_coe_file);
collision_data = load_coe_file(collision_coe_file, 2);

% Define collision map dimensions
collision_width = 45; % Collision grid width
collision_height = length(collision_data) / collision_width;

if mod(length(collision_data), collision_width) ~= 0
    error('Collision data dimensions do not align with specified width.');
```

```

end

fprintf('Collision Map Dimensions: %dx%d\n', collision_width, collision_height);

% Reshape collision data
collision_map = reshape(collision_data, [collision_width, collision_height]);

% Step 3: Resize Collision Map
fprintf('Resizing collision map to match map dimensions...\n');
collision_resized = imresize(collision_map, [map_height, map_width], 'nearest');

% Validate size of resized collision map
[resized_height, resized_width] = size(collision_resized);
fprintf('Resized Collision Map Dimensions: %dx%d\n', resized_width,
    ↳ resized_height);

if resized_height ~= map_height || resized_width ~= map_width
    warning('Resized collision map dimensions do not exactly match map
    ↳ dimensions. Adjusting...');
    collision_resized = collision_resized(1:min(map_height, resized_height),
    ↳ 1:min(map_width, resized_width));
    collision_resized(map_height, map_width) = 0; % Pad with zeros if smaller
end

% Step 4: Superimpose Collision Data
fprintf('Superimposing collision data on the map...\n');

% Create an overlay where unwalkable areas are highlighted in red
overlay_red = map_image_rgb; % Start with the original map
overlay_red(repmat(collision_resized == 1, [1, 1, 3])) = 0; % Set red channel to
    ↳ 0
overlay_red(:, :, 1) = overlay_red(:, :, 1) + uint8(collision_resized) * 255; %
    ↳ Add red highlights

% Step 5: Save and Display the Image
imshow(overlay_red);
title('Map with Collision Overlay');
imwrite(overlay_red, output_image);
fprintf('Overlay image saved to: %s\n', output_image);

%% Function to Load COE File
function data = load_coe_file(filename, radix)
    fileID = fopen(filename, 'r');
    if fileID == -1
        error('Could not open the COE file.');
```

```

    end

```

```

    % Parse COE file

```

```

    data_section = false;

```

```

    data = [];

```

```

    while ~feof(fileID)

```

```

line = strtrim(fgetl(fileID));
if contains(line, 'memory_initialization_vector')
    data_section = true;
    continue;
end
if data_section
    line = erase(line, ';');
    line = strrep(line, ',', ' ');
    if radix == 2
        values = sscanf(line, '%1d');
    elseif radix == 16
        values = sscanf(line, '%x');
    end data = [data; values];
end end fclose(fileID);
end

```

